



Министерство науки и высшего образования Российской
Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования

«Московский государственный технический университет имени
Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Компьютерные системы и сети (ИУ-6)»

ОТЧЕТ О ВЫПОЛНЕНИИ ДОМАШНЕЙ РАБОТЫ

по дисциплине «Математические методы анализа
данных и принятия решений»

Студент: Козлов Владимир Михайлович

Группа: ИУ6-13М

Тип задания: домашняя работа

Тема: Байесовский классификатор

Студент

подпись, дата

Козлов В.М.

Фамилия, И.О.

Преподаватель

подпись, дата

Фамилия, И.О.

Москва, 2025

бла бла

Оглавление

Введение	5
1 Исследовательская часть.....	6
1.1 Современные методы оптимизации распределения (offline partitioning) - METIS.....	6
1.1.1 Алгоритмы свёртки	7
1.1.2 Алгоритмы распределения	8
1.1.3 Алгоритмы уточнения	9
1.1.4 Экспериментальное сравнение	10
1.2 LIRA	11
1.2.1 Проблема длинного хвоста в распределении ближайших соседей	12
1.2.2 Концепция редундантности	13
1.2.3 Алгоритм дублирования	13
1.2.4 Использование обучаемой модели для управления редундантностью	15
1.2.5 Оценка эффективности	16
1.3 TAPER.....	19
1.3.1 Постановка задачи и целевая функция	19
1.3.2 Экстраверсия и интроверсия вершин	19
1.3.3 Представление нагрузки запросов: TPSTrie	20
1.3.4 От TPSTrie к Visitor Matrix	21
1.3.5 Алгоритм улучшения распределения	22
1.3.6 Оценка эффективности	23
1.4 Loom	25
1.4.1 Выявление мотивов запросов	25
1.4.2 Обнаружение мотивов в потоке обновлений графа	27
1.4.3 Размещение мотивов по разделам с учётом балансировки	28
1.4.4 Оценка эффективности	30
1.5 Методы потокового распределения (online partitioning)	31
1.5.1 Балансировочные	31
1.5.2 Хеширование	31
1.5.3 Streaming Graph Partitioning	32
1.5.4 Fennel алгоритм	35

2	Конструкторская часть.....	38
2.1	Постановка задачи распределения вершин графа	38
2.2	Структурная оптимизация.....	39
2.2.1	Структура проекта и основные зависимости	39
2.2.2	Классы для представления графовых структур (graph.hpp)	40
2.2.3	Структура класса Storage	44
2.2.4	Класс оптимизатора (optimizer.hpp)	49
2.3	Streaming Graph Partitioning.....	53
2.4	Алгоритм Fennel	54
2.4.1	Объективная функция	54
2.4.2	Инкрементальное вычисление и формула для δg	54
2.4.3	Структура проекта и основные зависимости	55
2.4.4	Классы для представления графовых структур (graph.hpp)	56
2.4.5	Структура класса FennelStorage	58
	Заключение.....	61
	Список использованных источников.....	62

Введение

1 Исследовательская часть

1.1 Современные методы оптимизации распределения (offline partitioning) - METIS

METIS (от MEtis Tournament Inspired Strategy) [1] - это набор программных библиотек и утилит, разработанных в Университете Миннесоты, для разбиения больших неориентированных графов и разрежения матриц.

Ключевая идея METIS - многоуровневая парадигма (Multilevel Paradigm). Вместо того чтобы работать с огромным исходным графом напрямую, METIS последовательно его упрощает, находит разбиение для маленького графа и затем интерполирует это разбиение обратно на исходный граф.

Алгоритм состоит из трёх этапов:

1. Схлопывание/Упрощение/Свёртка (англ. Coarsening) - уменьшение графа схлопыванием вершин в граф G_l .

Исходный граф G_0 последовательно преобразуется во всё более мелкие графы $G_1, G_2, \dots, G_l$. На каждом шаге пары смежных вершин "схлопываются/сворачиваются" в одну супервершину.

2. Распределение (англ. Partitioning) - распределение (разбиение) графа G_l . Поскольку граф G_l очень мал, для его разбиения можно использовать даже очень дорогие (вычислительно сложные) алгоритмы.

3. Развёртка (англ. Uncoarsening) - развёртка графа G_l , то есть действие обратное первому этапу. Также происходит улучшение (англ. refinement) - оптимизация разбиения в процессе развёртки. Порядок следующий:

(a) Разбиение, найденное для G_l , проецируется на граф G_{m-1} . Так как каждая вершина в G_{m-1} соответствовала вершине в G_l , разбиение определяется автоматически.

(b) Улучшение, то есть переброс вершин между шардами для получения лучшего распределения.

(c) Первые два пункта повторяются пока не будет получено разбиение для G_0 .

Этап хорошо подвергается параллелизации.

1.1.1 Алгоритмы свёртки

Свёртка в Metis сводится к поиску максимального паросочетания, но не наибольшего, так как это слишком затратное действие. и дальнейшего схлопывания рёбер паросочетания.

В базовом методе рассматриваются 4 алгоритма поиска максимального паросочетания:

1. Случайное (англ. Random Matching - RM) - берётся случайное ребро, затем случайное ребро, несмежное с ним, после чего ребро не связанное с выбранными ранее, и так далее пока таких рёбер не останется. Метод прост и эффективен для локальной задачи, но для уменьшения общей мощности разрезов в дальнейшем подходит плохо.

2. Паросочетание тяжёлых рёбер (англ. Heavy Edge Matching - НЕМ) - выбор рёбер максимального веса по очереди. Экспериментально было установлено, что это приводит к хорошему уменьшению общей мощности разрезов. В Metis этот алгоритм используется как основной.

3. Паросочетание лёгких рёбер (англ. Light Edge Matching - LEM) - выбор рёбер минимального веса по очереди. Алгоритм приводит к большему общему весу рёбер у свёрнутого графа, что может положительно влиять на качество улучшения некоторыми алгоритмами.

4. Паросочетание тяжёлых клик (англ. Heavy Clique Matching - HCM) - объединяет вершины, максимизируя плотность ребер создаваемого подграфа. В отличие от НЕМ, который выбирает тяжелые ребра, HCM оценивает, насколько две вершины (точнее вершины, которые свернулись в них) близки к формированию клики. Для вершин u и v вычисляется плотность рёбер (англ. edge density) по формуле:

$$\text{density}(u, v) = \frac{2(ce(u) + ce(v) + ew(u, v))}{(vw(u) + vw(v))(vw(u) + vw(v) - 1)}$$

, где $ce(u)$, $ce(v)$ - веса уже схлопнутых рёбер в вершинах, $ew(u, v)$ - вес ребра между u и v , $vw(u)$, $vw(v)$ - сумма степеней вершин, схлопнутых в данные.

Алгоритм схож с НЕМ, но учитывает и вершины изначального графа.

1.1.2 Алгоритмы распределения

Этап распределения является решением поставленной в секции 1 задачи для свёрнутого графа. В Metis рекурсивно используются алгоритмы бисекции графа, поэтому число шардов k должно быть степенью двойки, другие k приведут к плохой балансировке.

На этом этапе также рассматриваются 4 алгоритма:

1. Спектральная бисекция (англ. Spectral Bisection - SB) - Вычисляет собственный вектор Фидлера y матрицы Лапласиана $Q = D - A$, где:

$$a_{ij} = \begin{cases} ew(v_i, v_j) & \text{если } (v_i, v_j) \in E_m \\ 0 & \text{иначе} \end{cases}$$
$$d_{ii} = \sum_{(v_i, v_j) \in E_m} ew(v_i, v_j)$$

Разбиение: $P[j] = 1$ если $y_j \leq r$, и $P[j] = 2$ в противном случае, где r - выбранная взвешенная медиана y_j .

2. Алгоритм Кернигана-Лина (англ. Kernighan-Lin - KL) - алгоритм, работающий на идее улучшения разбиения путём обмена вершин между шардами. Для этого определена следующая функция улучшения распределения:

$$g_v = \sum_{(v,u) \in E \wedge P[v] \neq P[u]} w(v, u) - \sum_{(v,u) \in E \wedge P[v] = P[u]} w(v, u)$$

при положительном g_v перемещение вершины v приведёт к улучшению распределения. За одну итерацию алгоритм проходит по всем вершинам и пытается переместить все, итерации прерываются как только какое перемещение не приведёт к улучшению, либо когда будет достигнуто предельное число операций. Эксперименты показали, что для достаточно хорошего распределения как правило хватает 5-10 итераций.[1]. Для избегания работы "вхолостую" итерация прерывается при 50 перемещениях без улучшения, что хорошо показало себя при экспериментах. Также стоит отметить, что ещё можно ограничить алгоритм на минимальное кол-во перемещений за итераций.

Функция улучшения может быть рассчитана для всех вершин в начале итерации и при перемещении вершины обновляться только для соседей переме-

щённой вершины для оптимизации.

3. Алгоритм роста графа (англ. Graph Growing - GGP) - выбирается случайная вершина и от неё как при поиске в ширину обходятся вершины, формируя подграф, пока не будет набрана ровно половина вершин. Этот подграф будет первым шардом, остальные вторым. Далее рекомендуется применить KL для улучшения, что не займёт много времени в силу малого размера свёрнутого графа. Также стоит отметить, что вполне можно делить не пополам, а на другие равные части и применять KL к шардам попарно.

4. Жадный алгоритм роста графа (англ. Greedy Graph Growing - GGGP) - так как для предыдущего алгоритма и так рекомендуется применять KL после бисекции можно сразу при обходе графа идти по вершинам, которые будут давать наибольшее улучшение.

1.1.3 Алгоритмы уточнения

В Metis предлагаются два схожих алгоритма:

1. Алгоритм уточнения Кернигана-Лина - предлагается продолжать использовать KL на каждом новом развёрнутом графе G_i . Так как G_{i+1} Уже был хорошо распределён большого кол-ва перемещений не потребуется. Эксперименты показали, что на первой итерации получается наибольшее улучшение, поэтому применение только одной итерации выделяют как алгоритм KL(1) как значительном менее затратный, чем KL.

2. Граничный алгоритм улучшения Кернигана-Лина (англ. Boundary Kernighan-Lin Refinement - BKL) - так как итерации прерываются большинство вычислений тратятся впустую, особенно в KL(1), поэтому предлагается применять KL только к "граничным" вершинам, то есть к тем, которые смежны с вершинами из другого шарда. Это позволяет сильно уменьшить вычислительные затраты, что в свою очередь позволяет чаще делать несколько итераций, что ведёт к идее алгоритма BKL(*,1) - применять BKL пока вершин немного и перейти на BKL(1), когда их станет слишком много.

1.1.4 Экспериментальное сравнение

Граф	RM		HEM		LEM		HCM	
	без у.	с у.	без у.	с у.	без у.	с у.	без у.	с у.
BCSSTK31	25.28	7.82	14.67	8.03	71.98	7.38	20.16	7.77
BCSSTK32	18.70	7.25	15.09	7.04	69.10	7.07	15.63	7.30
BRACK2	20.70	5.65	14.50	5.78	51.24	5.59	18.94	5.40
CANT	41.69	16.48	24.87	16.48	83.34	16.48	26.60	16.48
COPTER2	19.63	9.18	26.44	8.78	59.14	9.17	16.93	8.93
CYLINDER93	29.25	11.10	16.06	11.10	82.48	11.14	19.82	11.42
4ELT	8.45	3.98	6.62	4.13	9.61	4.34	8.77	4.10
INPRO1	18.37	7.01	16.77	6.73	73.47	6.85	12.65	7.00
ROTOR	22.33	5.84	16.75	5.51	64.05	5.63	14.87	5.71
SHELL93	16.12	3.65	10.25	3.53	62.40	3.55	11.18	3.60
TROLL	12.76	3.70	9.40	3.57	57.56	3.55	10.29	3.64
WAVE	22.58	6.93	20.08	6.80	70.38	6.69	18.19	6.71

Таблица -2: Качество конечного распределения (λ) различных алгоритмов свёртки без и с улучшением для 32 шардов. [1]

Граф	SBP		GGP		GGGP	
	λ	IRTime	λ	IRTime	λ	IRTime
BCSSTK31	7.87	2.74	7.66	1.43	7.68	1.40
BCSSTK32	7.59	2.25	7.39	1.70	6.93	1.58
BRACK2	6.08	3.45	5.92	2.42	5.63	2.38
CANT	16.60	4.18	16.64	4.32	16.46	3.46
COPTER2	8.98	3.41	9.07	1.91	8.68	1.88
CYLINDER93	11.46	2.71	11.30	2.08	11.34	2.01
4ELT	3.90	1.00	3.90	0.72	3.82	0.70
INPRO1	7.07	2.35	6.82	1.59	6.70	1.61
ROTOR	5.58	4.75	6.02	4.30	5.49	3.36
SHELL93	3.97	6.01	3.64	5.02	3.57	4.89
TROLL	3.71	7.82	3.54	6.37	3.64	5.92
WAVE	7.06	7.18	6.78	4.84	6.79	4.57

Таблица -1: Конечное качество распределения алгоритмов начального распределения и общая скорость выполнения (начальное распределение ITime + улучшение BKL(1) RTime = IRTime) для 32 шардов. Результаты KL опущены в силу сильного их проигрыша по времени и качеству начального распределения остальным. [1]

Граф	KL(1)		KL	
	λ	RTime	λ	RTime
BCSSTK31	7.90	1.05	8.18	2.33
BCSSTK32	6.73	1.39	7.22	2.89
BRACK2	6.13	2.04	5.66	4.92
CANT	16.50	3.30	16.35	6.82
COPTER2	8.90	2.24	8.86	5.42
CYLINDER93	11.25	1.95	11.19	4.32
4ELT	4.00	0.44	4.00	0.96
INPRO1	6.77	1.28	6.79	3.41
ROTOR	5.84	4.98	5.86	13.09
SHELL93	3.96	9.27	3.44	52.40
TROLL	3.70	9.55	3.61	27.40
WAVE	7.03	8.72	6.83	19.36

Таблица 0: Качество и время улучшения для KL алгоритмов улучшения [1]

Граф	BKL(1)		BKL		BKL(*,1)	
	λ	RTime	λ	RTime	λ	RTime
BCSSTK31	8.08	0.76	7.86	1.91	8.03	1.27
BCSSTK32	7.31	0.96	6.94	2.27	7.04	1.47
BRACK2	5.67	1.16	5.40	3.21	5.78	2.36
CANT	16.58	2.43	16.30	5.49	16.48	3.16
COPTER2	9.10	1.12	8.66	3.11	8.78	1.83
CYLINDER93	11.14	1.40	11.14	2.98	11.08	1.88
4ELT	4.42	0.29	4.13	0.66	4.13	0.66
INPRO1	6.83	0.96	6.65	2.17	6.73	1.48
ROTOR	5.63	1.93	5.57	5.71	5.56	3.20
SHELL93	3.64	2.72	3.49	10.05	3.53	6.01
TROLL	3.66	4.14	3.50	13.12	3.57	5.84
WAVE	6.79	3.08	6.76	10.90	6.80	4.50

Таблица 1: Качество и время улучшения для BKL алгоритмов улучшения. [1]

1.2 LIRA

В данном разделе рассматривается фреймворк LIRA (Learning-based Query-aware Partition Framework), предложенный для решения проблем эффективности приближенного поиска ближайших соседей (ANN). Основное внимание уделяется ключевой идее авторов - использованию избыточности хранения (редундантности) для борьбы с неравномерностью распределения данных. [2]

1.2.1 Проблема длинного хвоста в распределении ближайших соседей

Классические методы ANN, основанные на жестком разбиении данных на шарды (hard partitioning), сталкиваются с фундаментальным ограничением, связанным с природой распределения данных в высокоразмерных пространствах. Даже при удачном выборе центроидов кластеров результаты запроса часто рассеяны по множеству шардов. Это явление, получившее название «длинный хвост» (long-tailed distribution), проявляется в том, что несколько соседей могут находиться на одном шарде, в то время как один-два соседа - в нескольких других.

Авторы провели серию экспериментов на реальных данных SIFT, чтобы подтвердить устойчивость этого феномена. Рисунок ?? демонстрирует, что при различном количестве шардов B (от 8 до 64) тысячи запросов из десяти тысяч имеют распределение, в котором хотя бы один шард содержит ровно одного ближайшего соседа.

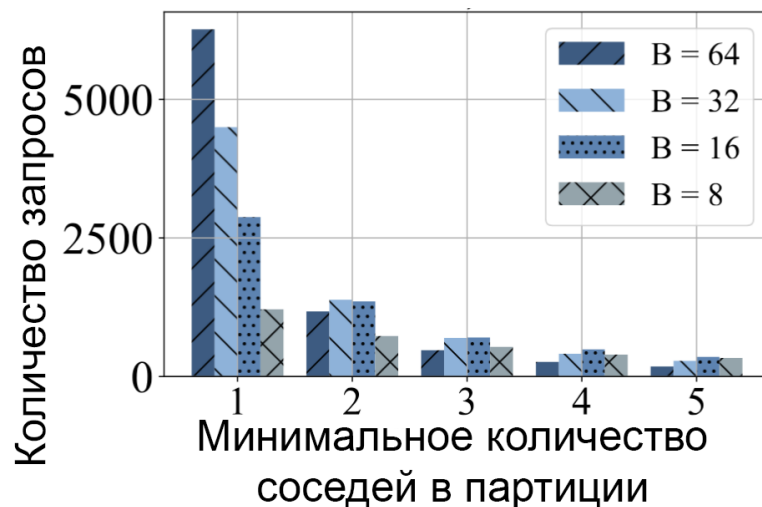


Рис. 1: Распределение запросов с длиннохвостыми kNN. Значительная часть запросов имеет хотя бы один шард с ровно одним соседом.

Этот, казалось бы, незначительный остаток (один вектор в шарде) оказывает непропорционально большое влияние на эффективность поиска. Если не учесть эту редкий шард, полнота поиска упадет. Если учесть - придется тратить ресурсы на просмотр шарда, где находится лишь один полезный результат. Именно для разрешения этой дилеммы и требуется пересмотр самого принципа организации данных.

1.2.2 Концепция редундантности

В ответ на ограничения жесткого разбиения авторы фреймворка LIRA предлагают отказаться от принципа «одна точка - один шард» и ввести элемент избыточности (redundancy) в структуру хранения. Ключевая идея заключается в том, чтобы стратегически дублировать определенные вектора данных, тем самым сглаживая длинный хвост распределения и уменьшая количество шардов, которые необходимо посетить для достижения высокой полноты поиска.

Цель редундантности можно проиллюстрировать простым примером. Пусть распределение kNN для некоторого запроса имеет вид $[5, 4, 1, 0, 0]$, что означает необходимость просмотра трех шардов. Если длиннохвостый вектор из третьего шарда продублировать в одну из двух первых, распределение изменится на $[5, 5, 0, 0, 0]$, и для покрытия всех k соседей потребуется просмотреть всего два шарда.

Однако реализация этой идеи сталкивается с двумя серьезными вызовами. Первый вызов связан с идентификацией векторов, которые являются длинным хвостом. Прямой подход требует вычисления kNN для всех точек данных, что имеет квадратичную сложность $O(N^2)$ и неприменим для больших данных. Второй вызов заключается в выборе шарда для дублирования: необходимо определить, в какой именно шард поместить копию вектора, чтобы это максимально сократило будущие запросы.

1.2.3 Алгоритм дублирования

Этап 1: Оценка вероятностей с помощью обученной модели.

Для всех векторов $v \in \mathcal{D}$ вычисляются предсказания обученной модели f , которая для каждого вектора (рассматриваемого как гипотетический запрос) возвращает вектор вероятностей:

$$\hat{p}^v = f(v, I_v) = [\hat{p}_1^v, \hat{p}_2^v, \dots, \hat{p}_B^v] \in [0, 1]^B$$

где $I_v = [dist(v, c_1), dist(v, c_2), \dots, dist(v, c_B)]$ - расстояния от вектора v до центроидов всех шардов, а $\hat{p}_b^v$ интерпретируется как вероятность того, что b -й шард содержит хотя бы одного из k ближайших соседей вектора v .

Этап 2: Оценка собственного probe для каждого вектора.

Для каждого вектора v вычисляется его собственное прогнозируемое количество шардов, необходимых для покрытия его k ближайших соседей:

$$\hat{n}_v^* = |\{b \mid \hat{p}_b^v > \sigma\}|$$

где σ - пороговое значение (по умолчанию 0.5). Величина $\hat{n}_v^*$ является оценкой того, насколько широко распределены ближайшие соседи самого вектора v по различным шардам.

Этап 3: Выявление длиннохвостых векторов-кандидатов.

Авторами эмпирически обнаружена корреляция: векторы с большим значением $\hat{n}_v^*$ с высокой вероятностью сами оказываются в длинном хвосте распределения kNN для других запросов. Формально это можно записать как:

$$P(v \in \text{LongTail}(q) \mid \hat{n}_v^* > \tau) \gg P(v \in \text{LongTail}(q))$$

где $\text{LongTail}(q)$ - множество векторов, являющихся единственными представителями своих шардов в результатах запроса q , а τ - некоторый порог.

На основе этой корреляции формируется множество кандидатов на дублирование:

$$\mathcal{D}_{\text{pick}} = \{v \in \mathcal{D} \mid \hat{n}_v^* \geq \text{Percentile}_\eta(\{\hat{n}_u^* \mid u \in \mathcal{D}\})\}$$

где Percentile_η - η -й процентиль распределения $\hat{n}^*$ по всему множеству данных, а η - гиперпараметр алгоритма (обычно 3-5%).

Этап 4: Выбор целевого шарда для дублирования.

Для каждого вектора-кандидата $v \in \mathcal{D}_{\text{pick}}$ необходимо выбрать целевой шард $b_{\text{target}}(v)$, в которую будет помещена его копия. Выбор основывается на следующем эмпирическом наблюдении: множество реплика-шардов $R(v)$ (шардов, в которые следует дублировать вектор для максимального сокращения будущих запросов) с высокой точностью покрывается top- M шардов по значению $\hat{p}_b^v$:

$$\text{Recall}_{\text{rep}}(M) = \frac{|\text{Top}_M(\hat{p}^v) \cap R(v)|}{|R(v)|} \approx 1 \quad \text{при умеренных } M$$

На практике используется следующее правило выбора:

$$b_{\text{target}}(v) = \arg \max_{b \neq b_{\text{orig}}(v)} \hat{p}_b^v$$

то есть выбирается шард с наибольшей предсказанной вероятностью среди всех шардов, кроме исходной.

В случае, если $\arg \max_b \hat{p}_b^v = b_{\text{orig}}(v)$ (что возможно, хотя исходный шард и имеет наибольшую вероятность), выбирается шард со вторым по величине значением $\hat{p}_b^v$:

$$b_{\text{target}}(v) = \arg \max_{b \neq b_{\text{orig}}(v)} \hat{p}_b^v = \operatorname{argmax}_{b \neq b_{\text{orig}}(v)} \hat{p}_b^v$$

Этап 5: Дублирование и обновление структуры данных.

Для каждого выбранного вектора $v \in \mathcal{D}_{\text{pick}}$ создается его копия v' (полностью идентичный вектор), которая добавляется в целевой шард $P_{b_{\text{target}}(v)}$:

$$P_{b_{\text{target}}(v)} \leftarrow P_{b_{\text{target}}(v)} \cup \{v'\}$$

Исходный вектор v при этом остается в своём шарде $P_{b_{\text{orig}}(v)}$. В результате общий объем данных увеличивается на $|\mathcal{D}_{\text{pick}}|$ векторов, что составляет $\eta\%$ от исходного размера.

1.2.4 Использование обучаемой модели для управления редундантностью

Для решения задач идентификации длиннохвостых векторов и выбора целевых шардов авторы предлагают использовать обучаемую модель. Обучение производится на подмножестве данных, где для каждого обучающего вектора известно, на каких шардах находятся его истинные ближайшие соседи. Модель учится предсказывать эту информацию для произвольных запросов.

В основе идентификации лежит эмпирически обнаруженная корреляция. Авторы заметили, что вектора с большим собственным $nprobe^*$ (количеством шардов, необходимых для покрытия их собственных kNN) с высокой вероятностью сами оказываются в длинном хвосте у других запросов. Поскольку вычислить истинный $nprobe^*$ для всех векторов вычислительно дорого, для его оценки используется предсказание обученной модели. Вектора с наибольшим предсказанным $nprobe$ отбираются в кандидаты на дублирова-

ние.

Для выбора целевого шарда также используется предсказание модели. Анализ показал, что шарды, в которые следует дублировать вектор, с высокой точностью входят в число тех, которым модель присваивает наибольшие вероятности для данного вектора. Таким образом, выбор целевого шарда также делегируется обученной модели.

Алгоритм дублирования для вектора v , исходно принадлежащего шарда b_{orig} , выглядит следующим образом. На основе предсказаний модели все шарды ранжируются по убыванию вероятности того, что они содержат ближайших соседей v . Выбирается шард b_{target} с наивысшим рангом, такая что $b_{target} \neq b_{orig}$. Вектор v дублируется в шард b_{target} . В случае, если наивысший ранг у b_{orig} , выбирается следующая по вероятности шарда.

Введение редундантности изменяет не только структуру хранения, но и сам процесс выполнения запросов. После того как избыточные шарды построены, для входящего запроса q выполняется следующий двухэтапный процесс.

На первом этапе обученная модель используется для предсказания того, какие шарды с наибольшей вероятностью содержат ближайших соседей запроса. Важно отметить, что благодаря предварительному дублированию длиннохвостых векторов, множество таких шардов оказывается меньше, чем в исходной структуре без редундантности.

На втором этапе поиск выполняется только внутри предсказанных шардов с использованием быстрых внутренних индексов (например, HNSW). Все найденные кандидаты собираются, сортируются по расстоянию до запроса, и из них выбираются k ближайших.

Таким образом, редундантность работает на опережение: еще на этапе построения индекса структура хранения адаптируется таким образом, чтобы будущие запросы требовали просмотра меньшего количества шардов. Это качественно меняет компромисс между полнотой поиска и количеством просматриваемых данных.

1.2.5 Оценка эффективности

Точность поиска.

Основной метрикой точности служит Recall@k - доля истинных k бли-

жайших соседей, найденных в результате поиска:

$$\text{Recall@k} = \frac{|S \cap S_{GT}|}{k} \times 100\%$$

где S - множество векторов, возвращенных алгоритмом, а S_{GT} - истинное множество k ближайших соседей запроса. Данная метрика отражает полноту поиска и является стандартной для задач ANN.

Вычислительная стоимость.

Для оценки эффективности поиска используются две взаимосвязанные метрики:

- **cmp** (distance computations) - общее количество вычислений расстояний между векторами, выполняемых в процессе поиска. Эта метрика непосредственно отражает вычислительную нагрузку и влияет на время выполнения запроса.
- **nprobe** (query fan-out) - количество шардов, просматриваемых при выполнении запроса. Данная метрика характеризует степень отсечения шардов и эффективность работы мета-индекса.

Пропускная способность.

Для оценки производительности в условиях двухуровневых индексов используется метрика QPS (queries per second) - количество запросов, обрабатываемых системой за секунду. Данная метрика интегрирует в себя как вычислительную стоимость отдельных запросов, так и влияние организации индекса на общую пропускную способность системы.

Результаты на малых датасетах. [2]

В таблице 2 представлено сравнение методов при фиксированной точности $\text{Recall@k} = 0.98$ для различных значений k на датасете SIFT. При $k = 100$ LIRA требует 96261 вычислений расстояний против 137276 у IVF (снижение на 29.8%) и просматривает 5.46 шардов против 8 у IVF (снижение на 31.6%). При увеличении k до 200 преимущество LIRA возрастает: снижение вычислений достигает 35.5%, а числа просматриваемых шардов - 37.1%. Это подтверждает, что предложенный подход особенно эффективен в условиях сильно выраженного длинного хвоста распределения, характерного для больших значений k .

тод	k = 100			k = 200		
	cmp	nprobe	Recall	cmp	nprobe	Recall
IVF	137276	8.00	0.98	153931	9.00	0.98
IVFFuzzy	144120	5.00	0.98	144120	5.00	0.98
BLISS	168778	9.00	0.98	168778	10.0	0.98
LIRA	96261	5.46	0.98	99279	5.66	0.98

Таблица 2: Сравнение производительности при Recall@k = 0.98 на датасете SIFT [2].

Результаты на крупных датасетах. [2]

Для оценки масштабируемости предложенного подхода авторы построили двухуровневые индексы, где LIRA выступает в роли мета-индекса, а HNSW используется для внутреннего поиска внутри шардов. В таблице 3 представлены результаты на трех крупных датасетах объемом 50 миллионов векторов каждый. Метод BLISS был исключен из сравнения на крупных датасетах из-за превышения доступного объема памяти при построении четырех групп двухуровневых индексов.

Датасет	Метод	QPS	Recall@100	nprobe
Deer	IVF	119	0.8996	32
	IVFFuzzy	170	0.9604	25
	LIRA	255	0.9655	15
BIGANN	IVF	104	0.9464	19
	IVFFuzzy	180	0.9607	23
	LIRA	257	0.9639	27
Yandex TI	IVF	160	0.7673	7
	IVFFuzzy	174	0.8316	25
	LIRA	246	0.8386	43

Таблица 3: Производительность двухуровневых индексов на крупных датасетах. [2]

На датасете Deer LIRA достигает 255 запросов в секунду, что на 114% выше, чем у IVF, и на 50% выше, чем у IVFFuzzy, при этом обеспечивая наилучшую точность (0.9655). На датасете BIGANN пропускная способность возрастает до 257 запросов в секунду (+147% к IVF, +43% к IVFFuzzy). Наиболее сложный датасет Yandex TI (200 измерений) также демонстрирует существенный прирост производительности: 246 запросов в секунду против 160 у IVF (+54%).

1.3 TAPER

В данном подразделе подробно рассматривается система TAPER (query-aware, partition-enhancement for large, heterogeneous, graphs), предложенная Хьюго Фертом и Паоло Миссье в работе [3]. Данное решение представляет собой фреймворк для инкрементального улучшения существующих распределений графов, который учитывает текущую нагрузку запросов и адаптируется к её изменениям. TAPER не создаёт распределение с нуля, а улучшает уже имеющееся, что делает его применимым в онлайн-режиме.

1.3.1 Постановка задачи и целевая функция

Авторы работы [3] ставят перед собой задачу улучшения k -непрерывного распределения $P_k(G)$ гетерогенного графа $G = (V, E, L_V, l)$, где $l(v)$ — метка вершины v , а L_V — множество меток. В качестве входных данных система принимает:

- существующее распределение $P_k^0(G)$ (например, полученное хешированием или алгоритмом Metis);
- поток запросов $\mathcal{Q} = \{(Q_1, n_1), \dots, (Q_h, n_h)\}$, где n_i — относительная частота запроса Q_i .

Основной метрикой качества распределения в контексте выполнения запросов является количество межшардовых переходов (inter-partition traversals, IPT). Снижение IPT напрямую ведёт к уменьшению задержек и сетевого трафика в распределённой системе. Цель TAPER — минимизировать суммарную вероятность межшардовых переходов, которую авторы называют экстраверсией (extroversion). Формально, результатом работы TAPER является новое распределение $P'_k(G, \mathcal{Q})$, улучшенное с учётом заданной нагрузки.

1.3.2 Экстраверсия и интроверсия вершин

Для количественной оценки вклада каждой вершины в межшардовые переходы авторы вводят понятия интроверсии и экстраверсии. Эти показатели базируются на вероятностной модели обхода графа, учитывающей паттерны запросов.

Пусть задан путь $p = v_{i_1} \rightarrow \dots \rightarrow v_{i_k} \rightarrow v$, соответствующий одному из паттернов запросов. Вероятность этого пути $Pr(p)$ может быть вычислена

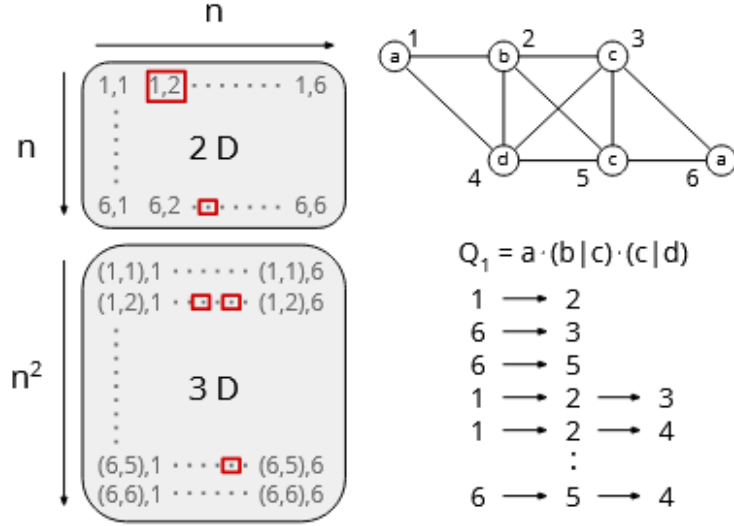


Рис. 2: Структура Visitor Matrix (VM) для хранения вероятностей путей различной длины.

как произведение условных вероятностей переходов. Интроверсия вершины $v \in V_i$ (где V_i — её родной шард) определяется как суммарная вероятность всех путей, ведущих к v , при условии, что следующий шаг также остаётся внутри V_i :

$$\text{introversion}(v) = \frac{1}{Pr(v)} \sum_{p \in \text{paths}(v, V_i)} \left(Pr(p) \cdot \sum_{v' \in V_i} VM_i(t)[p, v'] \right),$$

где $VM_i(t)$ — матрица посещений (Visitor Matrix) для шарда i , хранящая вероятности переходов для путей длины до t , а $Pr(v)$ — нормировочный коэффициент (суммарная вероятность всех путей, ведущих к v). Экстраверсия, соответственно, является дополнением до единицы:

$$\text{extroversion}(v) = 1 - \text{introversion}(v).$$

Именно экстраверсия служит для TAPER сигналом к перемещению вершины.

1.3.3 Представление нагрузки запросов: TPSTrie

Прямое вычисление VM имеет экспоненциальную сложность $O(|V|^t)$ и неприменимо для больших графов. Для решения этой проблемы авторы предлагают компактную структуру данных — префиксное дерево обобщения паттернов обхода (Traversal Pattern Summary Trie, TPSTrie). Идея заключается

в том, чтобы хранить не вероятности для конкретных вершин, а вероятности для последовательностей меток, которые требуются в запросах.

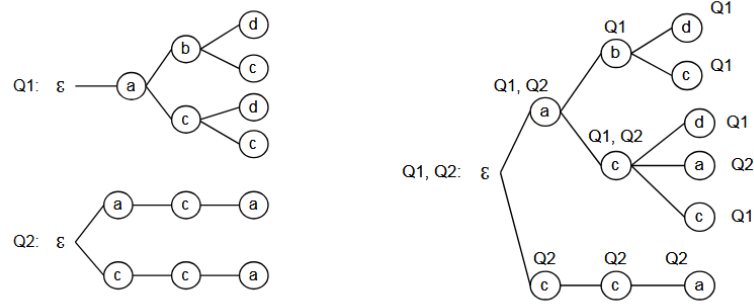


Рис. 3: Построение TPSTrie путём разбора регулярных выражений запросов и объединения результатов.

Каждый узел TPSTrie соответствует некоторой последовательности меток и помечается множеством запросов, которые могут породить такую последовательность. Каждому узлу ставится в соответствие вероятность $p(n)$, вычисляемая на основе частот запросов и структуры дерева. Например, для корневого перехода $\mathcal{E} \rightarrow a$ вероятность равна:

$$Pr(\mathcal{E} \rightarrow a) = \sum_{Q_i \in \mathcal{Q}} Pr(\mathcal{E} \rightarrow a | Q_i) \cdot Pr(Q_i),$$

где условные вероятности определяются количеством возможных первых шагов в запросе Q_i . При изменении частот запросов вероятности в узлах дерева пересчитываются, что позволяет TPSTrie адаптироваться к динамике нагрузки.

1.3.4 От TPSTrie к Visitor Matrix

Для получения вероятностей переходов между конкретными вершинами (т.е. заполнения VM) используется следующий подход. Для текущей вершины v_k и истории пути p выполняется поиск соответствующего узла в TPSTrie. Полученный узел указывает, какие метки допустимы для следующего шага, и с какими вероятностями. Затем эти вероятности равномерно распределяются среди всех соседей вершины v_k , имеющих данную метку. Это даёт вектор вероятностей перехода к каждому соседу, который и является строкой VM. Такой подход позволяет избежать хранения вероятностей для

всех возможных путей, заменяя их компактным представлением на уровне меток.

Для дальнейшего снижения вычислительной и пространственной сложности авторы вводят эвристики:

- Вершины, у которых нет внешних соседей, автоматически считаются безопасными ($\text{introversion} = 1$) и исключаются из рассмотрения.
- Вычисление экстраверсии вершины прекращается досрочно, если её накопленное значение интроверсии превышает заданный порог безопасности. Это позволяет сосредоточить ресурсы только на наиболее проблемных вершинах.

1.3.5 Алгоритм улучшения распределения

Процесс улучшения распределения в TAPER является итеративным и основан на жадном алгоритме рефининга, схожем с KL/FM (Kernighan-Lin / Fiduccia-Mattheyses), но с использованием экстраверсии в качестве целевой функции. Одна итерация (вызов TAPER) состоит из нескольких внутренних шагов (обычно 6-8), на каждом из которых выполняются следующие действия:

1. Для каждого шарда строится очередь кандидатов — вершин с наибольшей экстраверсией.
2. Для вершины-кандидата v определяется её семья (family). Это множество вершин, которые с высокой вероятностью участвуют в тех же путях, что и v (т.е. являются источниками путей к v с вероятностью выше 0.5). Перемещение всей семьи вместе с кандидатом повышает эффективность обмена.
3. Для семьи вычисляется предпочтительный целевой шард V_j (как правило, тот, где находится большинство соседей, участвующих в общих путях).
4. Происходит двухсторонний обмен (offer/receive):
 - Исходный шард оценивает потерю интроверсии при удалении семьи.
 - Целевой шард оценивает потенциальный выигрыш в интроверсии от добавления семьи.
 - Если выигрыш целевой шард превышает потерю исходной, обмен считается кооперативным и выполняется.
5. Если обмен отклонён, кандидату предлагаются другие соседние шар-

ды в порядке убывания предпочтительности.

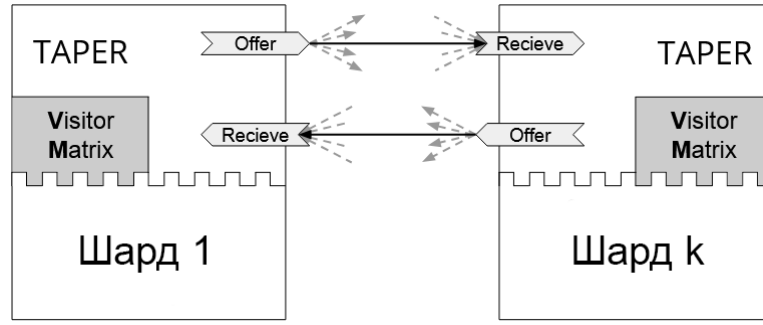


Рис. 4: Схема кооперативного обмена вершинами между шардами.

Каждая вершина может быть перемещена не более одного раза за внутреннюю итерацию. После обработки всех кандидатов запускается следующая внутренняя итерация. Процесс повторяется до тех пор, пока не будет достигнуто целевое улучшение или максимальное число итераций, при этом строго контролируется баланс размеров шардов (допустимое отклонение, например, 5%).

1.3.6 Оценка эффективности

Экспериментальная оценка, проведённая авторами на графах MusicBrainz (10М вершин) и синтетическом ProvGen (1М вершин), подтверждает эффективность предложенного подхода. Ключевые результаты, демонстрирующие преимущества TAPER, представлены ниже.

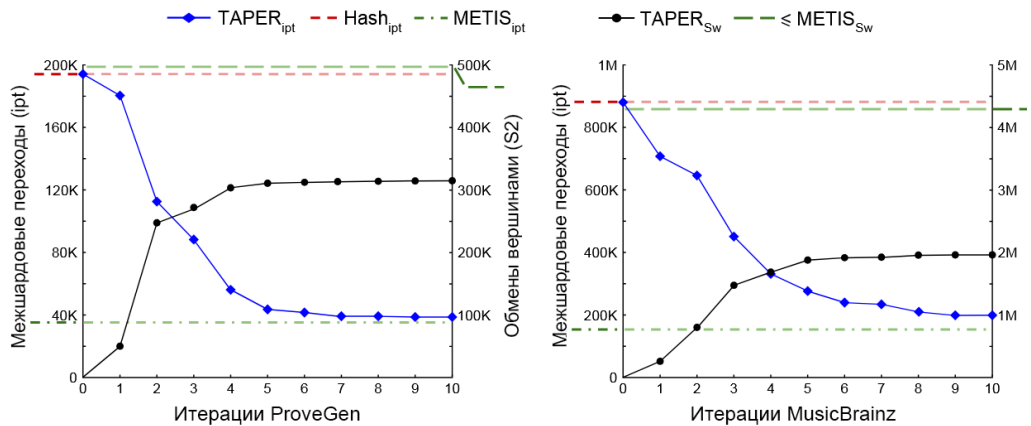


Рис. 5: Снижение количества межшардовых переходов (IPT) в процессе внутренних итераций TAPER. [3]

1. **Улучшение простых распределений:** При старте с распределения на основе хеширования, TAPER снижает количество IPT в среднем на

80% (см. рис. 5), достигая уровня качества, сопоставимого с Metis, всего за 6-8 итераций.

2. Улучшение качественных распределений: Даже при старте с распределения, полученного с помощью Metis (которое само по себе является эталоном качества по метрике min edge-cut), TAPER обеспечивает дополнительное снижение IPT в среднем на 30%. Это доказывает, что учёт нагрузки запросов позволяет находить более эффективные распределения, нежели чисто структурные методы.

3. Адаптация к изменению нагрузки: При изменении профиля запросов качество ранее оптимизированного распределения закономерно падает. Однако периодический запуск TAPER (см. рис. 6) позволяет восстанавливать высокую производительность, удерживая количество IPT значительно ниже уровня, характерного для статического распределения.

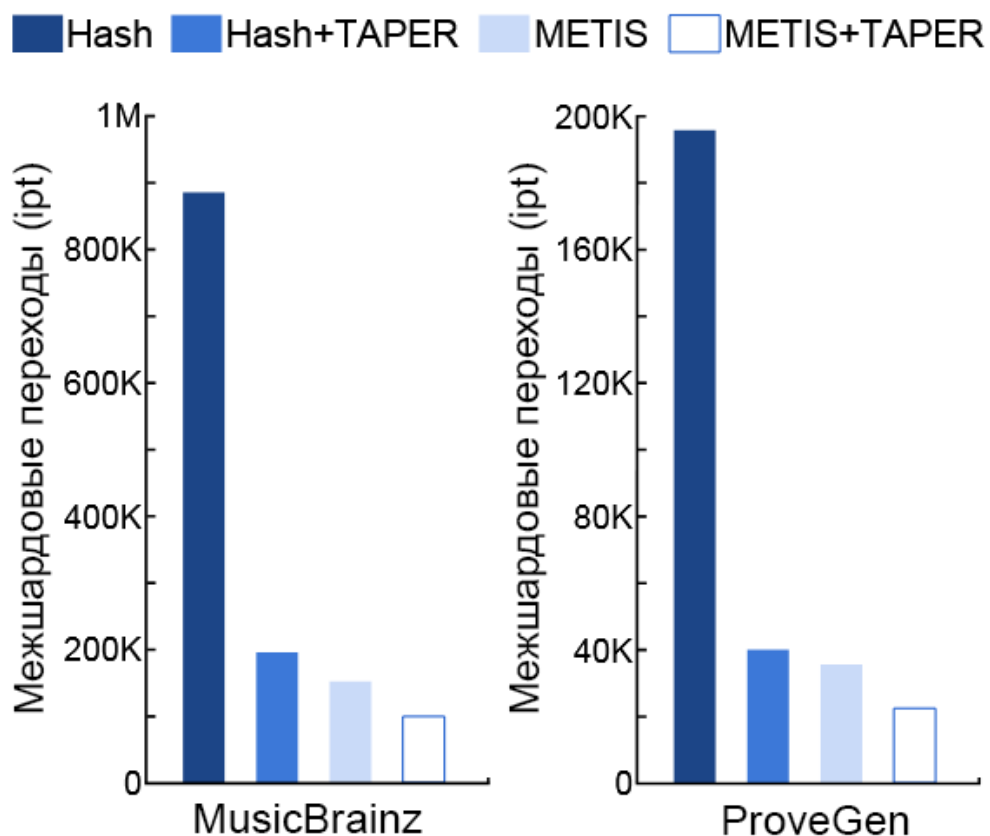


Рис. 6: Поддержание качества распределения при изменяющейся нагрузке с помощью периодических запусков TAPER. [3]

Авторы отмечают, что достигнутое улучшение требует значительно меньших накладных расходов по сравнению с полным перестроением графа. Количество перемещаемых вершин (объём коммуникации) при работе

TAPER примерно вдвое меньше, чем потребовалось бы для полного перестроения с нуля с помощью Metis.

Таким образом, система TAPER предлагает эффективное и практичное решение для задачи оптимизации хранения графовых данных. Её ключевые преимущества — учёт рабочей нагрузки, инкрементальность, адаптивность и доказанная способность значительно улучшать существующие распределения — делают её привлекательной для использования в реальных распределённых графовых СУБД.

1.4 Loom

В данном подразделе представлено подробное описание архитектуры и алгоритмов, предложенных авторами в статье [4]. Изложение разбито на три логические части, соответствующие основным этапам работы системы: выявление частотных структур в нагрузке, обнаружение этих структур в потоке графа и их размещение по разделам с учётом балансировки.

1.4.1 Выявление мотивов запросов

Первый этап работы Loom заключается в анализе заданного множества запросов Q с целью выявления наиболее часто встречающихся подграфов, которые авторы называют мотивами. Для этого предложена специализированная структура данных — TPSTry++ (Traversal Pattern Summary Trie), которая представляет собой направленный ациклический граф, кодирующий все возможные подграфы всех запросов из рабочей нагрузки.

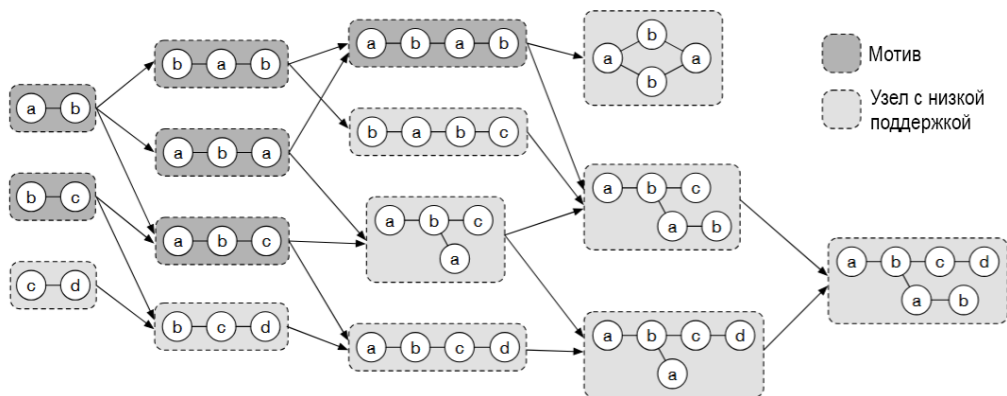


Рис. 7: Пример структуры TPSTry++

Каждый узел в TPSTry++ представляет собой уникальный подграф и хранит два ключевых атрибута: его сигнатуру (компактное числовое представление) и значение поддержки p . Поддержка узла равна суммарной частоте вхождения данного подграфа во все запросы из Q . Мотивом считается любой узел, чья поддержка превышает заданный порог $\mathcal{T}$. Например, для порога $\mathcal{T} = 40\%$ мотивами будут все подграфы, встречающиеся более чем в 40% запросов.

Для эффективного сравнения подграфов и выявления изоморфизмов при построении TPSTry++ авторы используют вероятностный метод, основанный на числовых сигнатурах, расширяющий подход, предложенный в работе [5]. Сигнатура графа $G_q = (V_q, E_q)$ вычисляется следующим образом:

1. Каждой метке вершины $l \in L_V$ ставится в соответствие случайное простое число $r(l)$ из диапазона $[1, p)$, где p — заданное простое число.
2. Для каждого ребра $e = (v_i, v_j)$ вычисляется реберный фактор:

$$\text{edgeFac}(e) = (r(f_l(v_i)) - r(f_l(v_j))) \mod p$$

3. Для каждой вершины v со степенью n вычисляется фактор степени:

$$\text{degFac}(v) = \prod_{k=1}^n ((r(f_l(v)) + k) \mod p)$$

4. Итоговая сигнатура графа есть произведение всех реберных факторов и факторов степеней всех вершин:

$$\text{sig}(G_q) = \left(\prod_{e \in E_q} \text{edgeFac}(e) \right) \cdot \left(\prod_{v \in V_q} \text{degFac}(v) \right)$$

Ключевым преимуществом такого подхода является его инкрементальность. Сигнатуру большего подграфа можно получить умножением сигнатуры меньшего подграфа на факторы, соответствующие добавленным рёбрам и вершинам. Это свойство активно используется на следующих этапах. Вероятность коллизий (когда два неизоморфных графа имеют одинаковую сигнатуру) регулируется выбором простого числа p . Авторы показали, что при $p = 251$ вероятность коллизий для графов с числом рёбер до 16 является

пренебрежимо малой.

1.4.2 Обнаружение мотивов в потоке обновлений графа

Второй этап работы Loom выполняется в реальном времени по мере поступления потока рёбер, формирующих динамический граф G . Система поддерживает скользящее окно фиксированного размера t (в рёбрах), которое служит временным буфером. Новые рёбра сначала попадают в это окно, и лишь при выходе из него окончательно распределяются по постоянным разделам. Такая задержка позволяет накапливать контекст для принятия более взвешенных решений о размещении. Само окно авторы рассматривают как временный раздел P_{temp} , что обеспечивает доступ к новым данным для запросов.

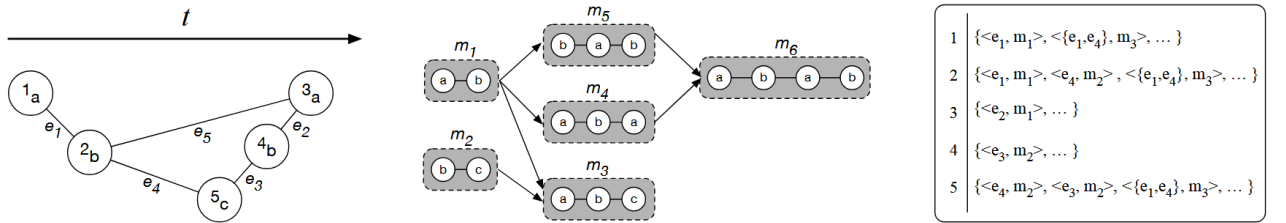


Рис. 8: Иллюстрация процесса сопоставления: Слева направо показаны: скользящее окно с рёбрами $e_1 \dots e_5$, набор мотивов из TPSTry++ ($m_1 \dots m_6$) и содержимое matchList, где каждому мотиву сопоставлены обнаруженные подграфы.

Процесс обнаружения мотивов в окне организован следующим образом:

1. При поступлении нового ребра $e = (v_1, v_2)$ вычисляется его сигнатура. Если она совпадает с сигнатурой однорёберного мотива в корне TPSTry++, ребро добавляется в окно P_{temp} , а информация о новом совпадении заносится в структуру matchList. Эта структура отображает каждую вершину на множество обнаруженных мотивов, которым она принадлежит. Если ребро не совпадает ни с одним мотивом, оно немедленно распределяется в постоянный раздел, минуя окно.

2. Если новое ребро инцидентно другим рёбрам, уже находящимся в окне, его добавление может привести к образованию более крупных подграфов, совпадающих с мотивами. Для проверки этого система использует matchList. Для каждого существующего подграфа g_i , связанного с вершина-

ми v_1 или v_2 , вычисляются факторы $\text{factors}(e, g_i)$, которые изменили бы его сигнатуру при добавлении e .

3. Затем проверяется, существует ли в TPSTry++ дочерний узел мотива, соответствующего g_i , набор факторов которого отличается от родительского ровно на $\text{factors}(e, g_i)$. Если такой узел существует и является мотивом, значит, новый подграф $g_i + e$ также является мотивом, и это совпадение добавляется в `matchList`.

4. Аналогичная проверка выполняется и для возможности объединения двух уже существующих в окне мотивов в один, более крупный, если они соединены через общую вершину.

Благодаря такому подходу `matchList` всегда содержит актуальную информацию обо всех подграфах в окне, которые являются мотивами, что служит основой для принятия решений о распределении на следующем этапе.

1.4.3 Размещение мотивов по разделам с учётом балансировки

Заключительный этап алгоритма вступает в силу, когда окно P_{temp} заполняется и новое ребро вытесняет из него самое старое ребро e . Именно в этот момент для ребра e и всех связанных с ним мотивов принимается решение об окончательном размещении в одном из k постоянных разделов S_i . Цель — разместить весь кластер рёбер, образующий мотив, внутри одного раздела, минимизируя тем самым будущие межраздельные переходы.

Для достижения этой цели авторы предлагают комбинацию двух эвристик. Для рёбер, не входящих в мотивы и потому не попавших в окно, применяется известная эвристика LDG (Linear Deterministic Greedy) [6]. Она назначает ребро в раздел S_i , максимизируя взвешенную сумму:

$$\max_{S_i \in P_k(G)} N(S_i, e) \cdot \left(1 - \frac{|\mathcal{V}(S_i)|}{C}\right),$$

где $N(S_i, e)$ — число инцидентных ребру e вершин, уже находящихся в разделе S_i , $\mathcal{V}(S_i)$ — текущее число вершин в разделе, а C — ограничение на ёмкость раздела. Второй множитель играет роль штрафа за переполнение, обеспечивая балансировку.

Для рёбер, которые являются частью мотивов и покидают окно, разработана новая, более сложная эвристика Equal Opportunity. Пусть M_e —

множество мотивов из `matchList`, содержащих ребро e . Эвристика не просто пытается разместить все эти мотивы в разделе с наибольшим числом общих вершин, но делает это более избирательно, учитывая как поддержку самих мотивов, так и необходимость глобальной балансировки.

Для каждого раздела S_i и каждого мотива $\langle E_k, m_k \rangle \in M_e$ вычисляется ставка (`bid`):

$$\text{bid}(S_i, \langle E_k, m_k \rangle) = \mathcal{N}(S_i, E_k) \cdot \left(1 - \frac{|\mathcal{V}(S_i)|}{C}\right) \cdot \text{supp}(m_k),$$

где $\mathcal{N}(S_i, E_k)$ — количество вершин из множества рёбер E_k , уже принадлежащих разделу S_i , а $\text{supp}(m_k)$ — поддержка мотива m_k в `TPSTry++`.

Ключевым нововведением является использование ратионирующей функции $l(S_i)$. Эта функция ограничивает "жадность" алгоритма, определяя, какая часть мотивов из M_e будет участвовать в формировании итоговой ставки для раздела S_i и, в случае его победы, какая часть будет ему назначена. $l(S_i)$ вычисляется на основе относительного размера раздела:

$$l(S_i) = \frac{|\mathcal{V}(S_i)|}{\min_{S \in P_k(G)} |\mathcal{V}(S)|} \cdot \alpha,$$

где α — параметр, управляющий агрессивностью ратионирования (авторы используют $\alpha = 2/3$). Функция устроена так, что меньшие разделы получают большее значение $l(S_i)$ и, следовательно, могут участвовать в торгах за большее число мотивов, что способствует выравниванию нагрузки.

Итоговое решение о размещении принимается на основе максимизации суммы ставок по первым $l(S_i) \cdot |M_e|$ мотивам в множестве M_e , отсортированным по убыванию поддержки $\text{supp}(m_k)$:

$$\max_{S_i \in P_k(G)} \sum_{k=0}^{l(S_i) \cdot |M_e|} \text{bid}(S_i, \langle E_k, m_k \rangle).$$

Такой подход гарантирует, что:

- Мотивы с наивысшей поддержкой (наиболее часто встречающиеся в запросах) имеют приоритет при размещении.
- Уже переполненные разделы ограничены в возможности получить новые, даже очень "ценные кластеры рёбер, что сохраняет общий баланс си-

стемы.

- Рёбра, которые не были назначены победившему разделу, остаются в окне P_{temp} , получая шанс быть назначенными позже вместе с другими, более релевантными для них мотивами.

1.4.4 Оценка эффективности

В ходе экспериментальной оценки авторы сравнили Loom с хешированием (baseline), а также с потоковыми партиционером LDG и Fennel [7]. В качестве основной метрики качества использовалось количество межраздельных переходов (IPT) при выполнении реалистичной нагрузки запросов на разделах, построенных каждым из методов.

Метод	IPT (отн. Hash)	Скорость (рёбер/с)	Зависимость от порядка
Hash	100% (базовый уровень)	—	Нет
LDG	$\approx 45\%$	$\approx 100k$	Средняя
Fennel	$\approx 20\text{-}25\%$ (лучше LDG)	$\approx 70k$	Средняя
Loom	15-40% (лучше Fennel)	$\approx 42\text{-}72k$	Высокая

Таблица 4: Сравнительная эффективность алгоритмов распределения. [4]

Результаты, обобщённые в Таблице 4, демонстрируют значительное превосходство Loom над существующими решениями:

- Loom стабильно обеспечивает на 20–25% меньшее количество межраздельных переходов по сравнению с Fennel, который является лучшим из workload-agnostic алгоритмов. В отдельных случаях, например, на гетерогенном графе MusicBrainz при BFS-порядке поступления данных, улучшение достигало 42%.

- Преимущество сохраняется для различного количества разделов (от 2 до 32).

- Основная плата за повышение качества — чувствительность к порядку поступления данных (особенно к случайному) и некоторое снижение скорости работы (в 2-3 раза медленнее Fennel). Однако даже в медленном режиме Loom обрабатывает более 40 000 рёбер в секунду, что, по мнению авторов, достаточно для большинства практических онлайн-сценариев.

- Увеличение размера скользящего окна позволяет значительно снизить негативное влияние случайного порядка поступления.

Таким образом, предложенное решение Loom успешно демонстрирует

применимость и высокую эффективность подхода, учитывающего информацию о запросах при потоковом распределении динамических графов.

1.5 Методы потокового распределения (online partitioning)

В этой секции будут использоваться следующие обозначения: P^t - распределение вершин по шардам (состояние шардов) в момент времени t , $P^t(i)$ - состояние шарда i в момент времени t , v - некая вершина, $N(v)$ - соседи вершины v , C - максимальная вместимость шарда.

1.5.1 Балансировочные

Простейшие методы распределения новых вершин, не решающие задачу, поставленную выше, а предназначенные просто для равномерного распределения данных по шардам. Стоят упоминания, так как используются в последующих методах, а также могут быть использованы на начальных этапах.

Всего их три:

1. Поочерёдное - распределение записей по шардам по очереди. По сути round-robin.
2. Групповое - записи на шарды загружаются сразу большим набором исходя из ожидаемого количества и следуемого из этого количества записей на каждом шарде.
3. Случайное/Псевдослучайное - абсолютно или частично случайное распределение вершин по шардам.

1.5.2 Хеширование

Технически попадает в группу методов описанную выше, конкретно в псевдослучайные, но, в отличие от них, активно используется сам по себе, а не как начальный этап или основа для других методов.

Заключается в использовании хеш-функции на каких-либо атрибутах вершин. Для лучшего решения задачи можно хешировать атрибуты, которые естественным образом совпадают у близких вершин, например, географические.

Используется, например, в ArangoDB [8] (Файл `arangod\Sharding\ShardingStrategyDefault.cpp`, метод `getResponsibleShard`), где хеширование применяется дважды: сначала хешируются атрибуты, после чего хешируется полученный хеш. Примечание: это не связано с алгоритмом двойного хеширования для решения коллизий, коллизии в понимании этого алгоритма не случаются при распределении вершин по шардам.

1.5.3 Streaming Graph Partitioning

Потоковое распределение графов [6] (англ. Streaming Graph Partitioning) - алгоритм, а точнее набор алгоритмов, для распределения вершин графа, которые приходят "потоком" по одной или небольшими наборами (оба варианта простейшим образом можно переводить друг в друга, поэтому далее их разделения не будет), с использованием эвристик для оценки насколько новая вершина подходит и штрафов, чтобы не избежать загрузки большинства вершин на один шард.

Метод учитывает три возможных варианта организации потока:

1. Случайное - вершины графа приходят в случайном порядке.
2. Обход в ширину (BFS) - первая вершина выбирается случайно, каждая следующая приходит согласно обходу в ширину.
3. Обход в глубину (DFS) - аналогично обходу в ширину, но согласно обходу в глубину.

Для балансировки распределения в эвристиках используются следующие весовые функции:

1. $w(t, i) = 1$ - без взвешивания.
2. $w(t, i) = 1 - \frac{|P^t(i)|}{C}$ - линейный вес.
3. $w(t, i) = 1 - e^{|P^t(i)-C|}$ - экспоненциальный вес.

Предлагаемые эвристики:

1. Балансировка - распределение по шардам с наименьшим кол-вом вершин.
2. Групповая (Chunking) - описанное выше групповое распределение
3. (Вес) Детерминированная жадная (DG):

$$ind = \operatorname{argmax}_{i \in k} \{|P^t(i) \cap N(v)|w(t, i)\}$$

где $w(t, i)$ - одна из описанных выше весовых функций.

4. (Вес) Случайная жадная (RG): выбор шарда происходит следующего распределения

$$Pr(i) = \frac{|P^t(i) \cap N(v)|w(t, i)}{Z}$$

где $w(t, i)$ - одна из описанных выше весовых функций, а Z - нормализующая константа. В оригинальной статье отмечается, что случайная эвристика рассматривается только потому что она может лучше работать в худшем случае организации потока.

5. (Вес) Треугольники:

$$ind = \operatorname{argmax}_{i \in k} \left\{ \frac{|E(P^t(u) \cap N(v), (P^t(u) \cap N(v)))|}{\binom{|P^t(u) \cap N(v)|}{2}} \right\} w(t, i)$$

где $w(t, i)$ - одна из описанных выше весовых функций, $E(S, T)$ - набор рёбер между наборами верши S и T . Идея заключается в сохранении наибольшего кол-ва треугольников на шарде, что, очевидно, приведёт к хорошей связности на нём.

6. Балансировка большого При возможности определять степень вершины и насколько она велика относительно всего графа распределять вершины с большой степенью балансировкой, а с маленькой DG.

Дальнейшие эвристики предполагают буферизацию:

7. Предпочтение большому - начать с вершин больших степеней, использовать балансировку большого.

8. Избегание большого - распределить жадным алгоритмом вершины малых степеней, когда останутся только вершины с большими степенями распределить их DG.

9. Жадный EvoCut - использовать EvoCut [9] на подграфе в буфере, найти малые множества (англ. nibbles) с хорошей проводимостью и распределить их по шардам DG.

Метод в первую очередь направлен на графы соц.сетей, где и используется.

Сравнение эвристик

Для оценки качества определения считается процент среднего улучше-

ния по следующей формуле:

$$\text{gain} = \frac{\text{edges_cut_random} - \text{edges_cut_heuristic}}{\text{edges_cut_random} - \text{edges_cut_METIS}} \times 100\%$$

Где edges_cut_random - кол-во рёбер между шардами при хешировании, $\text{edges_cut_heuristic}$ - кол-во рёбер между шардами в эксперименте, а edges_cut_METIS количество рёбер между шардами при использовании METIS.

Метод и его производный Fennel [7] используются в соц. сетях и рекомендательных системах, на графах которых и будет проводиться сравнение.

Сравнение эвристик представлено в следующей таблице:

Сокращение	Эвристика	<i>BFS</i>	<i>DFS</i>	<i>Случайное</i>
B	Балансировка	-1.5	-1.3	-0.2
C	Групповое	37.6	35.7	0.7
H	Хеширование	-1.9	-2.1	-1.7
DG	Детерминированная жадная	57.7	54.7	45.4
RG	Случайная жадная	45.5	44.9	38.7
T	Треугольники	49.7	48.4	40.2
LDG	Лин. Дет. жадная	76.0	73.0	75.3
LRG	Лин. Случ. жадная	46.4	44.9	39.1
LT	Лин. Треугольники	55.4	54.6	49.3
EDG	Эксп. Дет. жадная	59.4	56.2	47.5
ERG	Эксп. Случ. жадная	45.6	45.6	38.8
ET	Эксп. Треугольники	50.7	49.3	41.6
BB	Балансировка большого	67.8	68.5	63.3
PB	Предпочтение большому	-9.5	-18.6	-23.1
AB	Избегание большого	-27.3	-38.6	-46.4
GE	Жадный EvoCut	60.3	58.6	43.1

Таблица 5: Среднее улучшение каждой эвристики по всем наборам данных и размерам партиций [6]

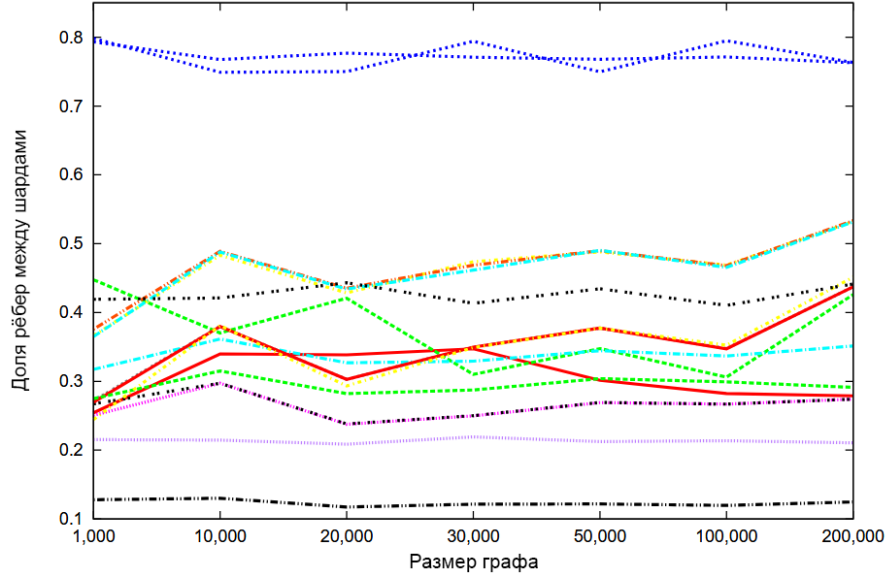


Рис. 9: Визуализация доли рёбер между шардами для 4 шардов с порядком BFS. Нижняя линия - METIS, предпоследняя нижняя - Лин. Дет. жадная [6]

Как можно увидеть по таблице и графику выше LDG даёт наилучшие результаты.

1.5.4 Fennel алгоритм

Fennel алгоритм [7] - модификация потокового распределения графов.

Модификация заключается в определении так называемой объективной функции k -распределения $g(P)$, значение которой должно максимально увеличиться при распределении новой вершины. По своей сути функция является эвристикой метода потокового распределения графов.

Опуская детали вывода функции описанные в оригинальной статье объективная функция k -распределения имеет следующий вид:

$$g(P) = \sum_{i=1}^k [e(S_i, S_i) - p \binom{|S_i|}{2}]$$

где $p = 2\alpha = m^{\frac{k\gamma-1}{n^\gamma}}$, $1 \leq \gamma \leq 2$.

Несмотря на относительную громоздкость самой функции, расчёт всех её вариантов не требуется, так как нужно найти максимальное $\delta g(v, S_i) = g(S_1, S_2, \dots, S_i \cup v, \dots, S_k) - g(S_1, S_2, \dots, S_k)$ для $i \in \{1 \dots k\}$, притом что все $g(S_1, S_2, \dots, S_k)$ и $e(S_j, S_j) - p \binom{|S_j|}{2}$ были найдены при заносе предыдущих вершин. Также, очевидно, каждое $g(S_1, S_2, \dots, S_i \cup v, \dots, S_k)$ может быть вычислено

на соответствующем шарде, если он способен на вычисления, что позволяет распараллелить все расчёты.

Балансировка

Стоит отдельно упомянуть балансировку, так как алгоритм в "сыром" виде совсем не гарантирует хорошую балансировку, а при $\gamma = 1$, когда алгоритм сводится к отправлению вершин в шард с наибольшим числом её соседей, вообще имеет тенденцию к плохой балансировке.

Поэтому также предложена надстройка, позволяющая гарантировать балансировку для $\rho = \tau$. Она заключается в изначальном отметании шардов, добавление вершины на которую превысит эту нагрузку, то есть таких шардов, где $|S_i \cup v| > \tau \frac{n}{k}$.

Сравнение с другими методами по качеству

Ниже предоставлены сравнения результатов работы алгоритма с другими эвристиками при $\gamma = \frac{3}{2}$ и $\tau = 1.1$.

Метод	BFS		Random	
	λ	ρ	λ	ρ
H	96.9%	1.01	96.9%	1.01
B	97.3%	1.00	96.8%	1.00
LDG	34%	1.01	40%	1.00
EDG	39%	1.04	48%	1.01
T	61%	2.11	78%	1.01
LT	63%	1.23	78%	1.10
ET	64%	1.05	79%	1.01
Fennel	14%	1.10	14%	1.02
METIS	8%	1.00	8%	1.02

Таблица 6: Качество работы различных методов на графе amazon0312 ($k = 32$) [7]

m	k	Fennel		METIS	
		λ	ρ	λ	ρ
7 185 314	4	62.5%	1.04	65.2%	1.02
6 714 510	8	82.2%	1.04	81.5%	1.02
6 483 201	16	92.9%	1.01	92.2%	1.02
6 364 819	32	96.3%	1.00	96.2%	1.02
6 308 013	64	98.2%	1.01	97.9%	1.02
6 279 566	128	98.4%	1.02	98.8%	1.02

Таблица 7: Результаты Fennel и METIS, усреднённые по 5 случайным графам, сгенерированным по модели HP(5 000, k, 0.8, 0.5) [7].

Сравнение с другими методами по времени

	Fennel		LDG		METIS	
Кластеров (k)	λ	ρ	λ	ρ	λ	ρ
2	6.8%	1.1	34.3%	1.04	11.98%	1.02
4	29%	1.1	55.0%	1.07	24.39%	1.03
8	48%	1.1	66.4%	1.10	35.96%	1.03

Таблица 8: Результаты работы методов при распределении графа Twitter (≈ 1.5 млрд рёбер). Распределение через Fennel и LDG заняло [7].

	Время выполнения [с]		Коммуникация [МБ]	
Кластеров (k)	Hash	Fennel	Hash	Fennel
4	32.27	25.49	321.41	196.9
8	17.26	15.14	285.35	180.02
16	10.64	9.05	222.28	148.67

Таблица 9: Средняя длительность шага и средний объем данных, передаваемых за шаг при выполнении [7]

2 Конструкторская часть

2.1 Постановка задачи распределения вершин графа

В контексте распределенных графовых баз данных задача распределения вершин (графового партиционирования) формулируется следующим образом. Пусть дан неориентированный граф $G = (V, E)$, где V - множество вершин, $|V| = n$, а E - множество рёбер, $|E| = m$. Требуется найти такое разбиение (распределение) $P = \{S_1, S_2, \dots, S_k\}$ множества вершин V на k непересекающихся подмножеств (шардов) S_i , что:

- $\bigcup_{i=1}^k S_i = V$.
- $S_i \cap S_j = \emptyset$ для всех $i \neq j$.

Качество распределения оценивается по двум основным критериям, которые необходимо оптимизировать:

1. **Минимизация разрезов (Edge Cut)**. Мощность разреза $\partial e(P)$ определяется как количество рёбер, концы которых оказались в разных шардах. Формально:

$$|\partial e(P)| = \sum_{i=1}^k |e(S_i, V \setminus S_i)|$$

Минимизация этого параметра напрямую влияет на снижение сетевого трафика при выполнении распределенных запросов.

2. **Балансировка нагрузки (Load Balancing)**. Неравномерное распределение вершин приводит к перегрузке одних узлов и простоем других. Для количественной оценки используется показатель нормализованной максимальной нагрузки ρ , который необходимо минимизировать:

$$\rho = \frac{\max_{i=1}^k (|S_i|)}{n/k}$$

Идеально сбалансированное распределение соответствует $\rho = 1$.

Таким образом, целевая задача заключается в поиске такого распределения P^* , которое минимизирует мощность разрезов $|\partial e(P)|$ при максимально допустимом значении дисбаланса $\rho \leq \tau$, где τ - заданный порог.

2.2 Структурная оптимизация

В рамках данной курсовой работы была реализована вычислительная модель для расчёта и оптимизации метрики улучшения распределения g_v для граничных вершин графа согласно алгоритму Кернигана-Лина. Реализация включает в себя четыре основных компонента:

1. Классы для представления графовых структур (вершины, рёбра, ключи).
2. Классы имитации шины для общения компонентов
3. Класс хранилища (Storage) для управления вершинами и их связями.
4. Класс оптимизатора (StorageOptimizer) для расчёта метрики g_v .

Основной задачей практической части являлось создание инфраструктуры для вычисления функции улучшения распределения, определенной в алгоритме Кернигана-Лина:

$$g_v = \sum_{\substack{(v,u) \in E \\ P[v] \neq P[u]}} w(v,u) - \sum_{\substack{(v,u) \in E \\ P[v] = P[u]}} w(v,u)$$

2.2.1 Структура проекта и основные зависимости

Языком разработки был выбран C++ в силу его низкоуровневости и скорости, а также с намерением в дальнейшем внедрить эти наработки в графовую БД на C++ научного руководителя.

Проект организован в виде набора заголовочных файлов (header files), что соответствует современным подходам разработки на C++. Основные файлы проекта:

- `graph.hpp` – содержит базовые классы для представления графовых структур.
- `interface_bus.hpp` – содержит интерфейс, описывающий основные сообщения в шине.
- `bus.hpp` – содержит простую реализацию имитации шины.
- `storage.hpp` – реализует класс хранилища для управления вершинами.
- `optimizer.hpp` – содержит реализацию оптимизатора с вычислением метрики g_v .

- `main.cpp` – демонстрационный файл с тестовым сценарием.

Ниже представлен релевантный для решения задачи практики код.

2.2.2 Классы для представления графовых структур (`graph.hpp`)

Класс `NodeKey`

Класс `NodeKey` представляет собой обёртку для ключа вершины графа. Он обеспечивает типобезопасность и возможность использования различных типов данных в качестве ключей (целые числа, строки и т.д.).

Листинг 1: Класс `NodeKey` в `graph.hpp`

```
1 template <typename KeyType>
2 class NodeKey {
3 public :
4     KeyType key_value;
5
6     NodeKey() : key_value(KeyType()) {};
7     NodeKey(const KeyType& key) : key_value(key) {};
8
9     // Оператор присваивания
10    NodeKey<KeyType>& operator=(const NodeKey<KeyType>& other) {
11        if (this != &other) {
12            key_value = other.key_value;
13        }
14        return *this;
15    };
16
17    // Дружественные операторы сравнения
18    friend bool operator<(const NodeKey<KeyType>& lhs, const
19        NodeKey<KeyType>& rhs) {
20        return lhs.key_value < rhs.key_value;
21    }
22
23    friend bool operator>(const NodeKey<KeyType>& lhs, const
24        NodeKey<KeyType>& rhs) {
25        return rhs < lhs;
26    }
27
28    friend bool operator==( const NodeKey<KeyType>& lhs, const
29        NodeKey<KeyType>& rhs) {
30        return lhs.key_value == rhs.key_value;
```



```

28 }
29
30 friend bool operator!=(const NodeKey<KeyType>& lhs, const
    NodeKey<KeyType>& rhs) {
31     return !(lhs == rhs);
32 }
33 };

```

Класс `NodeKey` является шаблонным, что позволяет использовать различные типы данных в качестве ключей вершин. Это важно для обеспечения гибкости при работе с различными типами графовых данных. Известно, что в конечной реализации используются строковые ключи, но была добавлена гибкость для потенциального использования пользовательских гибридных ключей.

Класс `Edge`

Класс `Edge` представляет ребро графа с весом и дополнительными параметрами.

Листинг 2: Класс `Edge` в `graph.hpp`

```

1 template <typename KeyType>
2 class Edge {
3 private:
4     float weight;
5     bool directional;
6     std::map<std::string, Parameter> parameters;
7     NodeKey<KeyType> from;
8     NodeKey<KeyType> to;
9 public:
10    float get_weight() const {
11        return weight;
12    }
13    bool is_directional() const {
14        return directional;
15    }
16    const NodeKey<KeyType>& get_from() const {
17        return from;
18    }
19    const NodeKey<KeyType>& get_to() const {
20        return to;
21    }

```

```

22
23     NodeKey<KeyType> get_other(const NodeKey<KeyType>& node)
const {
24         if (node == to) {
25             return from;
26         } else if (node == from) {
27             return to;
28         }
29
30         return node;
31     }
32
33     NodeKey<KeyType> get_other(NodeKey<KeyType>* node) const {
34         if (*node == to) {
35             return from;
36         } else if (*node == from) {
37             return to;
38         } else {
39             return *node;
40         }
41     }
42 };

```

Класс Node

Класс Node представляет вершину графа и содержит всю информацию о её связях с другими вершинами.

Листинг 3: Класс Node в graph.hpp (часть 1)

```

1 template <typename KeyType>
2 class Node {
3 private:
4     NodeKey<KeyType> key;
5 public:
6     std::map<std::string, Parameter> parameters;
7     std::map<NodeKey<KeyType>, Edge<KeyType>> edges;
8 }
9     NodeKey<KeyType> get_key() const {
10         return key;
11     }
12
13     void add_edge(Edge<KeyType> new_edge) {
14         NodeKey<KeyType> current_key = this->get_key();

```

```

15         edges[new_edge.get_other(current_key)] = new_edge;
16     }
17
18     void add_edges(std::map<NodeKey<KeyType>, Edge<KeyType>>
new_edges) {
19         edges.merge(new_edges);
20     }
21
22     bool remove_edge_to(NodeKey<KeyType> neighbour_key) {
23         return edges.erase(neighbour_key);
24     }
25
26     bool remove_edge(Edge<KeyType> removing_edge) {
27         NodeKey<KeyType> current_key = this->get_key();
28         return
edges.erase(removing_edge.get_other(current_key));
29     }
30 };

```

Класс имеет несколько конструкторов для удобного создания вершин с различными конфигурациями связей. Все рёбра хранятся в вершине, от ссылок было решено отказаться, так как в конечной реализации хранилища должны находиться на разных машинах, а в вершины периодически перемещаться между ними.

Класс интерфейса шины (interface_bus.hpp)

Крайне важный интерфейс, через который происходит взаимодействие всей системы. Описывает методы добавления, удаления и запроса вершин, а также объявлений о добавлении и удалении для возможности другим хранилищам знать где находится другая вершина.

И наконец ключевые для алгоритма Кернигана-Лина методы получения граничных вершин и рёбер `ask_neighbours_to_storage` и `ask_edges_to_storage`.

Листинг 4: Базовая структура класса IBus

```

1 template <typename KeyType>
2 class IBus {
3 public:
4     virtual Node<KeyType> request_node(const NodeKey<KeyType>&
node) = 0;

```

```

5     virtual int send_add_node(const Node<KeyType>& node) = 0;
6     virtual bool send_add_node(const Node<KeyType>& node, int
    storage_id) = 0;
7     virtual bool send_remove_node(const NodeKey<KeyType>& node)
    = 0;
8     virtual bool send_remove_node(const NodeKey<KeyType>& node,
    int storage_id) = 0;
9     virtual int ask_who_has(int asker_id, NodeKey<KeyType> key)
    = 0;
10    virtual void announce_add(NodeKey<KeyType> key, int
    storage_id, std::set<Edge<KeyType>> edges) = 0;
11    virtual void announce_remove(NodeKey<KeyType> key, int
    storage_id) = 0;
12    // запрашивает у source вершины, соседствующие с target
13    virtual std::set<Node<KeyType>>
    ask_neighbours_to_storage(int source, int target) = 0;
14    // запрашивает у source рёбра, идущие в target
15    virtual std::set<Edge<KeyType>> ask_edges_to_storage(int
    source, int target) = 0;
16 };

```

Класс шины SimpleBus (bus.hpp)

Простая синхронная однопоточная реализация методов IBus. Релевантный код слишком длинный для вставки в основную часть, поэтому представлен в приложении А.

Класс хранилища (storage.hpp)

Класс Storage представляет собой хранилище вершин графа и реализует логику управления внутренними и внешними связями.

2.2.3 Структура класса Storage

Листинг 5: Базовая структура класса Storage

```

1 template <typename KeyType>
2 class Storage {
3 private:
4     typedef Node<KeyType> StorageNode;
5     typedef NodeKey<KeyType> Key;
6     int storage_id;
7     std::map<Key, StorageNode> nodes;
8     // external_edges[storage edge go to][external node][local node] = edge

```

```

9 std::map<int, std::map<Key, std::map<Key, Edge<KeyType>>>>
    external_edges;
10 IBus<KeyType> *bus = nullptr;
11
12 public:
13 Storage(int id)
14     :storage_id(id) {}
15 Storage(int id, std::map<Key, StorageNode> _nodes)
16     :storage_id(id), nodes(_nodes) {}
17
18 int get_id() const { return storage_id; };
19 void connect_to_bus(IBus<KeyType>* _bus) { bus = _bus; };

```

Хранилище идентифицируется уникальным `storage_id` и содержит вершины в виде хэш-таблицы для обеспечения быстрого доступа по ключу.

Добавление вершин с автоматическим созданием связей

Листинг 6: Методы добавления вершин класса Storage

```

1 std::optional<std::set<Edge<KeyType>>> add_node(const
    StorageNode& node){
2     Key key = node.get_key();
3     std::cout << storage_id << " adding " << key.key_value <<
        std::endl;
4     typename std::map<Key, StorageNode>::iterator it =
        nodes.find(key);
5     if (it != nodes.end()) {
6         return std::nullopt;
7     }
8     nodes[key] = node;
9
10    std::set<Edge<KeyType>> external_edges_to_announce;
11    for (typename std::map<Key, Edge<KeyType>>::const_iterator
        edge_it = node.edges.begin(); edge_it != node.edges.end();
        ++edge_it) {
12        const Key& neighbor_key = edge_it->first;
13        const Edge<KeyType>& edge = edge_it->second;
14        std::cout << storage_id << " looks for " <<
            neighbor_key.key_value << std::endl;
15        // Ищем соседа в текущем хранилище
16        typename std::map<Key, StorageNode>::iterator it2 =
            nodes.find(neighbor_key);
17        if (it2 != nodes.end()) {

```

```

18         std::cout << storage_id << " node " <<
neighbor_key.key_value << " is inside , no asking " <<
std::endl;
19         // Сосед найден в этом же хранилище - добавляем обратное ребро
20         it2->second.add_edge(edge);
21     } else {
22         std::cout << storage_id << " node " <<
neighbor_key.key_value << " is outside , asking " <<
std::endl;
23         // Сосед не найден - спрашиваем у шины, где он находится
24         int neighbours_storage_id =
bus->ask_who_has(storage_id , neighbor_key);
25         std::cout << storage_id << " asked for " <<
neighbor_key.key_value << " answer: " <<
neighbours_storage_id << std::endl;
26         if (neighbours_storage_id != -1) {
27             // Сохраняем внешнее ребро
28
external_edges[neighbours_storage_id][neighbor_key][key] =
edge;
29             external_edges_to_announce.insert(edge);
30         }
31         // Если сосед нигде не найден - игнорируем (возможно, будет
добавлен позже)
32     }
33 }
34
35 return external_edges_to_announce;
36 };
37
38 bool add_node_and_announce(const StorageNode& node) {
39     if (bus == nullptr) {
40         return false;
41     };
42     std::optional<std::set<Edge<KeyType>>> external_edges =
add_node(node);
43     if (!external_edges.has_value()) {
44         return false;
45     }
46     bus->announce_add(node.get_key(), storage_id ,
external_edges.value());
47     return true;

```

48 };

Метод `add_node` не только добавляет вершину в хранилище, но и автоматически создает связи с её соседями, что обеспечивает целостность графовой структуры. Это требуется для будущей реализации потокового распределения. Также метод `add_node_and_announce` позволяет при добавлении объявить о добавлении новой вершины.

Удаление вершин с автоматическим удалением связей

Листинг 7: Метод удаления вершин класса `Storage`

```
1 bool remove_node(const Key& key) {
2     if (!has_node(key)) {
3         return false;
4     };
5     StorageNode node = nodes.find(key)->second;
6     nodes.erase(key);
7
8     for (typename std::map<NodeKey<KeyType>,
9         Edge<KeyType>>::iterator edge_it = node.edges.begin();
10        edge_it != node.edges.end(); ++edge_it) {
11         Key other_key = edge_it->second.get_other(key);
12         typename std::map<Key, StorageNode>::iterator it =
13         nodes.find(other_key);
14         if (it != nodes.end()) {
15             it->second.remove_edge_to(key);
16         }
17     }
18
19     return true;
20 };
```

```
19 bool remove_node_and_announce(const Key& key) {
20     if (remove_node(key)) {
21         bus->announce_remove(key, storage_id);
22         return true;
23     }
24     return false;
25 };
```

Методы `remove_node` и `remove_node_and_announce` аналогично их `add` версиям автоматически следят за целостностью графа как локально, так и

внешне. Это значительно снижает вычислительную сложность, так как исключает из рассмотрения вершины, не имеющие внешних связей.

Методы для работы с граничными вершинами

Для реализации граничного алгоритма Кернигана-Лина требуется уметь получать вершины, граничащие с другим хранилищем, чем занимается метод `get_nodes_with_neighbors_in_storage` используя карту внешних соседей.

Листинг 8: Метод для получения граничных вершин

```
1  std::set<StorageNode> result;
2
3  typename std::map<int, std::map<Key, std::map<Key,
   Edge<KeyType>>>>::const_iterator storage_it =
   external_edges.find(target_storage_id);
4  if (storage_it == external_edges.end()) {
5      return result;
6  }
7
8  const std::map<Key, std::map<Key, Edge<KeyType>>>&
   node_edges = storage_it->second;
9  for (typename std::map<Key, std::map<Key,
   Edge<KeyType>>>::const_iterator node_edges_it =
   node_edges.begin(); node_edges_it != node_edges.end();
   ++node_edges_it) {
10     const std::map<Key, Edge<KeyType>>& local_node_edges =
        node_edges_it->second;
11     for (typename std::map<Key,
        Edge<KeyType>>::const_iterator local_node_edges_it =
        local_node_edges.begin(); local_node_edges_it !=
        local_node_edges.end(); ++local_node_edges_it) {
12         Key local_key = local_node_edges_it->first;
13         typename std::map<Key, StorageNode>::const_iterator
        node_it = nodes.find(local_key);
14         if (node_it != nodes.end()) {
15             result.insert(node_it->second);
16         }
17     }
18 }
19
20 return result;
```

В листинге 9 представлен метод получения рёбер между хранилищами, что позволяет несколько упростить дальнейшие вычисления.

Листинг 9: Метод получения рёбер в другое хранилище

```

1 std::set<Edge<KeyType>> get_all_edges_to_storage(int
    target_storage_id) const {
2     std::set<Edge<KeyType>> result;
3     typename std::map<int, std::map<Key, std::map<Key,
        Edge<KeyType>>>>::const_iterator storage_it =
        external_edges.find(target_storage_id);
4     if (storage_it == external_edges.end()) {
5         return std::set<Edge<KeyType>>();
6     } else {
7         const std::map<Key, std::map<Key, Edge<KeyType>>>&
            external_nodes_map = storage_it->second;
8         for (typename std::map<Key, std::map<Key,
            Edge<KeyType>>>::const_iterator node_edges_it =
            external_nodes_map.begin(); node_edges_it !=
            external_nodes_map.end(); ++node_edges_it) {
9             const std::map<Key, Edge<KeyType>>& node_edge_map =
                node_edges_it->second;
10            for (typename std::map<Key,
                Edge<KeyType>>::const_iterator edges_it =
                node_edge_map.begin(); edges_it != node_edge_map.end();
                ++edges_it) {
11                result.insert(edges_it->second);
12            }
13        }
14    }
15    return result;
16 }

```

2.2.4 Класс оптимизатора (optimizer.hpp)

Класс `ExternalStorageOptimizer` является центральным компонентом реализации алгоритма Кернигана-Лина и отвечает за расчёт метрики улучшения g_v .

Структура класса оптимизатора

Листинг 10: Класс `StorageOptimizer`

```

1 template <typename KeyType>
2 class ExternalStorageOptimizer {
3 private:
4 IBus<KeyType>* bus;
5 public:
6 ExternalStorageOptimizer (IBus<KeyType>* _bus)
7     : bus(_bus) {}
8 }

```

Основной метод расчёта метрик

Метод `calculate_gvs()` демонстрирует практическую реализацию итерации Boundary KL (алгоритм работы только с граничными вершинами). Он получает граничные вершины из обоих хранилищ и вычисляет для каждой из них метрику улучшения g_v , а метод `calculate_gv` занимается непосредственно расчётом метрики.

Листинг 11: Методы расчёта g_v

```

1 private:
2 float calculate_gv(const Node<KeyType>& node,
   std::set<Edge<KeyType>> boundary_edges) const {
3     NodeKey<KeyType> this_key = node.get_key();
4     float internal_edges_weight = 0;
5     float external_edges_weight = 0;
6
7     std::map<NodeKey<KeyType>, Edge<KeyType>>
   boundary_edges_map;
8     for (typename std::set<Edge<KeyType>>::const_iterator
   edge_it = boundary_edges.begin();
9         edge_it != boundary_edges.end(); ++edge_it) {
10         NodeKey<KeyType> other = edge_it->get_other(this_key);
11         if (!(other == this_key)) {
12             boundary_edges_map[other] = *edge_it;
13         }
14     }
15
16     for (typename std::map<NodeKey<KeyType>,
   Edge<KeyType>>::const_iterator edge_it = node.edges.begin();
17         edge_it != node.edges.end(); ++edge_it) {
18         const NodeKey<KeyType>& neighbor_key = edge_it->first;
19         const Edge<KeyType>& edge = edge_it->second;

```

```

19         if (boundary_edges_map.find(neighbor_key) !=
boundary_edges_map.end()) {
20             external_edges_weight += edge.get_weight();
21         } else {
22             internal_edges_weight += edge.get_weight();
23         }
24     }
25
26     return internal_edges_weight - external_edges_weight;
27 }
28
29 public:
30 std::map<int, std::map<Node<KeyType>, float>> calculate_gvs(int
storage1, int storage2) const {
31     std::map<int, std::map<Node<KeyType>, float>> result;
32     // Получаем граничные вершины для обоих хранилищ
33     std::set<Node<KeyType>> boundary_nodes1 =
bus->ask_neighbours_to_storage(storage1, storage2);
34     std::set<Node<KeyType>> boundary_nodes2 =
bus->ask_neighbours_to_storage(storage2, storage1);
35
36     std::set<Edge<KeyType>> boundary_edges1 =
bus->ask_edges_to_storage(storage1, storage2);
37     std::set<Edge<KeyType>> boundary_edges2 =
bus->ask_edges_to_storage(storage2, storage1);
38
39     result[storage1] = std::map<Node<KeyType>, float>();
40     typename std::set<Node<KeyType>>::const_iterator it;
41     for (it = boundary_nodes1.begin(); it !=
boundary_nodes1.end(); ++it) {
42         const Node<KeyType>& node = *it;
43         result[storage1][node.get_key()] = calculate_gv(node,
boundary_edges1);
44     }
45
46     result[storage2] = std::map<Node<KeyType>, float>();
47     for (it = boundary_nodes2.begin(); it !=
boundary_nodes2.end(); ++it) {
48         const Node<KeyType>& node = *it;
49         result[storage2][node.get_key()] = calculate_gv(node,
boundary_edges2);

```

```

50     }
51     return result;
52 };

```

Метод оптимизации

Метод `optimize()` демонстрирует практическую реализацию оптимизации KL. Он итеративно пользуется алгоритмом Кернигана-Лина, получает вершины с негативным g_v , после чего перемещает их в другое хранилище.

Листинг 12: Метод оптимизации

```

1 void optimize(int storage1, int storage2, int iterations_limit
  = 5) {
2     if (iterations_limit == 0) return;
3
4     int iteration = 0;
5     std::map<int, std::set<Node<KeyType>>> negative_gvs =
      get_negative_gvs(calculate_gvs(storage1, storage2));
6     do {
7         typename std::map<int,
          std::set<Node<KeyType>>>::iterator map_it;
8         for (map_it = negative_gvs.begin(); map_it !=
          negative_gvs.end(); ++map_it) {
9             int this_storage = map_it->first;
10            int other_storage = this_storage == storage1 ?
              storage2 : storage1;
11            std::set<Node<KeyType>>& nodes = map_it->second;
12            typename std::set<Node<KeyType>>::const_iterator
              set_it;
13            for (set_it = nodes.begin(); set_it != nodes.end();
              ++set_it) {
14                const Node<KeyType>& node = *set_it;
15                Node<int> removed =
                  bus->request_node(node.get_key());
16                bus->send_remove_node(removed.get_key(),
                  this_storage);
17                bus->send_add_node(removed, other_storage);
18            }
19        }
20        ++iteration;
21        negative_gvs = get_negative_gvs(calculate_gvs(storage1,
          storage2));

```

```

22     } while (iteration < iterations_limit &&
        !negative_gvs.empty());
23 }

```

2.3 Streaming Graph Partitioning

Традиционные методы оптимизации распределения, такие как METIS, требуют полного знания структуры графа для выполнения разбиения и работают в пакетном (offline) режиме. Однако во многих современных приложениях графы динамически изменяются: новые вершины и рёбра поступают непрерывно. Для таких сценариев разработаны методы потокового распределения (online partitioning).

В модели потокового распределения [6] вершины графа поступают на вход системы последовательно (в виде потока). Алгоритм должен принять решение о размещении каждой новой вершины v в одном из k шардов немедленно, основываясь только на информации об уже распределённых вершинах и, возможно, на знании соседей $N(v)$ новой вершины. Это накладывает жесткие ограничения на вычислительную сложность, но позволяет обрабатывать графы произвольного размера.

В работе [6] рассматривается семейство эвристик для потокового партиционирования, которые используют весовую функцию для учета текущей загруженности шардов и структуры связей. Цель эвристик - максимизировать количество соседей новой вершины в том шарде, куда она помещается. Общая формула для выбора шарда i выглядит следующим образом:

$$\text{ind} = \arg \max_{i \in \{1..k\}} \{|P^t(i) \cap N(v)| \cdot w(t, i)\}$$

где $P^t(i)$ - множество вершин в шарде i в момент времени t , а $w(t, i)$ - весовая функция, отвечающая за балансировку (например, линейная $w(t, i) = 1 - |P^t(i)|/C$ или экспоненциальная, где C - максимальная вместимость шарда). Одной из наиболее эффективных эвристик, согласно экспериментальным данным, является линейная детерминированная жадная эвристика (LDG).

2.4 Алгоритм Fennel

Алгоритм Fennel [7] представляет собой развитие идей потокового распределения графов и предлагает более формализованный подход к построению эвристики. Вместо комбинирования дискретных метрик (количество соседей) и весовых функций, Fennel вводит непрерывную объективную функцию $g(P)$, характеризующую качество текущего распределения P .

2.4.1 Объективная функция

Авторы Fennel определяют объективную функцию для k -распределения следующим образом:

$$g(P) = \sum_{i=1}^k [e(S_i, S_i) - p \binom{|S_i|}{2}]$$

где:

- $e(S_i, S_i)$ - количество рёбер, оба конца которых лежат внутри шарда S_i (внутренние рёбра).
- $p = 2\alpha = m \frac{k^{\gamma-1}}{n^\gamma}$, $1 \leq \gamma \leq 2$.
- α - параметр, регулирующий важность балансировки.
- γ - параметр, определяющий форму штрафа за размер шарда. В оригинальной работе рекомендуется $\gamma = \frac{3}{2}$.

Логика функции проста: она поощряет размещение рёбер внутри одного шарда (увеличение $e(S_i, S_i)$) и штрафует за слишком большой размер шарда (член $-p \binom{|S_i|}{2}$), что способствует балансировке.

2.4.2 Инкрементальное вычисление и формула для δg

Ключевым преимуществом Fennel для потоковой обработки является то, что для принятия решения о размещении новой вершины v не нужно пересчитывать $g(P)$ целиком. Достаточно вычислить прирост $\delta g(v, S_i)$, который получит система, если вершина v будет добавлена в существующий шард S_i .

Если предположить, что соседи v уже распределены, то добавление v в S_i увеличит количество внутренних рёбер в этом шарде на число соседей v , уже находящихся в S_i , то есть на $|S_i \cap N(v)|$.

Штраф за размер шарда изменяется с $-p\binom{|S_i|}{2}$ на $-p\binom{|S_i|+1}{2}$. Таким образом, изменение $\delta g(v, S_i)$ для шарда S_i вычисляется по формуле:

$$\delta g(v, S_i) = |S_i \cap N(v)| - p\left(\binom{|S_i|+1}{2} - \binom{|S_i|}{2}\right) \quad (1)$$

Именно это значение и используется в качестве основной эвристики. Алгоритм выбирает для вершины v тот шард i , который максимизирует $\delta g(v, S_i)$. Такой подход позволяет производить все вычисления распределенно: каждый шард может независимо рассчитать δg для себя, после чего выбирается шард с максимальным значением, либо случайный среди максимальных равных.

В рамках данной курсовой работы был реализован модуль потокового распределения вершин и модифицированы реализованные ранее модули:

1. Классы имитации шины для общения компонентов
2. Класс Fennel-хранилища (FennelStorage), расширяющий обычное хранилище для поддержки алгоритма феннеля.

2.4.3 Структура проекта и основные зависимости

Языком разработки был выбран C++ в силу его низкоуровневости и скорости, а также с намерением в дальнейшем внедрить эти наработки в графовую БД на C++ научного руководителя.

Проект организован в виде набора заголовочных файлов (header files), что соответствует современным подходам разработки на C++. Основные файлы проекта:

- `graph.hpp` – содержит базовые классы для представления графовых структур.
- `interface_bus.hpp` – содержит интерфейс, описывающий основные сообщения в шине.
- `bus.hpp` – содержит простую реализацию имитации шины.
- `storage.hpp` – реализует класс хранилища для управления вершинами.
- `optimizer.hpp` – содержит реализацию оптимизатора с вычислением метрики g_v .
- `main.cpp` – демонстрационный файл с тестовым сценарием.

Ниже представлен релевантный для решения задачи практики код. Пол-

ный код представлен в приложении А.

2.4.4 Классы для представления графовых структур (graph.hpp)

Класс интерфейса шины (interface _bus.hpp)

Крайне важный интерфейс, через который происходит взаимодействие всей системы. Описывает методы добавления, удаления и запроса вершин, а также объявлений о добавлении и удалении для возможности другим хранилищам знать где находится другая вершина.

И наконец ключевые для алгоритма Кернигана-Лина методы получения граничных вершин и рёбер `ask_neighbours_to_storage` и `ask_edges_to_storage`.

Листинг 13: Базовая структура класса IBus

```
1 template <typename KeyType>
2 class IBus {
3 public:
4     virtual Node<KeyType> request_node(const NodeKey<KeyType>&
        node) = 0;
5     virtual int send_add_node(const Node<KeyType>& node) = 0;
6     virtual bool send_add_node(const Node<KeyType>& node, int
        storage_id) = 0;
7     virtual bool send_remove_node(const NodeKey<KeyType>& node)
        = 0;
8     virtual bool send_remove_node(const NodeKey<KeyType>& node,
        int storage_id) = 0;
9     virtual int ask_who_has(int asker_id, NodeKey<KeyType> key)
        = 0;
10    virtual void announce_add(NodeKey<KeyType> key, int
        storage_id, std::set<Edge<KeyType>> edges) = 0;
11    virtual void announce_remove(NodeKey<KeyType> key, int
        storage_id) = 0;
12    virtual float get_streaming_euristics_change(const
        Node<KeyType>& node, int storage_id) = 0;
13    // запрашивает у source вершины, соседствующие с target
14    virtual std::set<Node<KeyType>>
        ask_neighbours_to_storage(int source, int target) = 0;
15    // запрашивает у source рёбра, идущие в target
16    virtual std::set<Edge<KeyType>> ask_edges_to_storage(int
        source, int target) = 0;
```


17 };

Класс шины SimpleBus (bus.hpp)

Простая синхронная однопоточная реализация методов IBus. В данной работе важна реализация send_add_node, реализующий выбор хранилища для отправки вершины и вспомогательном метода get_streaming_euristics_change:

Листинг 14: Реализация метода get_streaming_euristics_change

```
1 int send_add_node(const Node<KeyType>& node) override {
2     std::vector<IStorage<KeyType>*> best_storages;
3     float best_euristics = -10000000;
4
5     for (typename std::map<int, IStorage<KeyType>*>::iterator
6         it = storages.begin(); it != storages.end(); ++it) {
7         float this_euristics =
8             it->second->get_streaming_euristics_change(node,
9                 total_edges);
10        if (this_euristics > best_euristics) {
11            best_storages.clear();
12            best_storages.push_back(it->second);
13            best_euristics = this_euristics;
14        } else if (this_euristics == best_euristics) {
15            best_storages.push_back(it->second);
16        }
17    }
18    if (best_storages.empty()) {
19        return -1;
20    } else {
21        std::uniform_int_distribution<> dist{0,
22            best_storages.size() - 1};
23        IStorage<KeyType>* random_storage =
24            best_storages[dist(gen)];
25        send_add_node(node, random_storage->get_id());
26        return random_storage->get_id();
27    }
28 };
```

```

27         return false;
28     }
29     return
        storages[storage_id]—>get_streaming_euristics_change(node,
        total_edges);
30 }

```

Как можно увидеть, метод опрашивает все хранилища и выбирает наилучшее. Если "лучших" окажется несколько (обычно если ни в одном хранилище нет соседей новой вершины), то выбирается случайный.

Класс хранилища (storage.hpp)

Класс `Storage` представляет собой хранилище вершин графа и реализует логику управления внутренними и внешними связями.

2.4.5 Структура класса FennelStorage

Для работы Феннель-алгоритму требуется знания о кол-ве шардов, а также балансирующее число. Для передачи этих данных используется структура `FennelParameters`, представленная в листинге 15.

Листинг 15: Структура `FennelParameters`

```

1 struct FennelParameters {
2     /*k*/int storage_number;
3     /*gamma*/float balancing_number;
4 };

```

Само же хранилище `FennelStorage` представлено в листинге 16

Листинг 16: Базовая структура класса `Storage`

```

1 template <typename KeyType>
2 class FennelStorage : public Storage<KeyType> {
3 protected:
4 typedef Node<KeyType> StorageNode;
5 typedef NodeKey<KeyType> Key;
6 FennelParameters params;
7 int number_of_nodes;
8 float streaming_euristics = 0;
9
10 float get_p_parameter(int number_of_nodes, long
        number_of_edges){
11     if (number_of_nodes == 0) {

```

```

12         return 0;
13     }
14     return 2 * number_of_edges * std::pow(params.storage_number
, (params.balancing_number - 1)) / std::pow(number_of_nodes,
params.balancing_number);
15 }
16
17 float fennel_function(const Node<KeyType>& node, long
total_edges) {
18     float p = get_p_parameter(number_of_nodes, total_edges);
19     float dg = 0;
20     for (typename std::map<Key, Edge<KeyType>>::const_iterator
edge_it = node.edges.begin(); edge_it != node.edges.end();
++edge_it) {
21         const Key& neighbor_key = edge_it->first;
22         const Edge<KeyType>& edge = edge_it->second;
23         // Ищем соседа в текущем хранилище
24         typename std::map<Key, StorageNode>::iterator it2 =
this->nodes.find(neighbor_key);
25         if (it2 != this->nodes.end()) {
26             ++dg;
27         }
28     }
29     dg -= p * this->size();
30     return dg;
31 }
32 public:
33
34 FennelStorage(int id, FennelParameters _params, std::map<Key,
StorageNode> _nodes = std::map<Key, StorageNode>())
35     : Storage<KeyType>(id, _nodes), params(_params) {}
36
37 float get_streaming_euristics_change(const Node<KeyType>& node,
long total_edges) override {
38     return fennel_function(node, total_edges);
39 }
40
41 void get_add_announcement(Key key, int announcer_id,
std::set<Edge<KeyType>> edges) override {
42     ++number_of_nodes;
43     Storage<KeyType>::get_add_announcement(key, announcer_id,

```

```
        edges);  
44 };  
45  
46 void get_remove_announcement(Key key, int announcer_id)  
    override {  
47     —number_of_nodes;  
48     Storage<KeyType>::get_remove_announcement(key,  
        announcer_id);  
49 }  
50 };
```

Заключение

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. *Karypis G., Kumar V.* A fast and high quality multilevel scheme for partitioning irregular graphs // SIAM Journal on Scientific Computing. — 1998. — Т. 20, № 1. — С. 359—392. — DOI: 10.1137/S1064827595287997.
2. LIRA: A Learning-based Query-aware Partition Framework for Large-scale ANN Search / X. Zeng [и др.] // Proceedings of the ACM on Web Conference 2025. — ACM, 04.2025. — С. 2729—2741. — (WWW '25). — DOI: 10.1145/3696410.3714633. — URL: <http://dx.doi.org/10.1145/3696410.3714633>.
3. *Firth H., Missier P.* TAPER: query-aware, partition-enhancement for large, heterogenous, graphs. — 2016. — arXiv: 1603.04626 [cs.DB]. — URL: <https://arxiv.org/abs/1603.04626>.
4. *Firth H., Missier P., Aiston J.* Loom: Query-aware Partitioning of Online Graphs. — 2017. — arXiv: 1711.06608 [cs.DB]. — URL: <https://arxiv.org/abs/1711.06608>.
5. Event pattern matching over graph streams / C. Song [и др.] // Proc. VLDB Endow. — 2014. — Дек. — Т. 8, № 4. — С. 413—424. — ISSN 2150-8097. — DOI: 10.14778/2735496.2735504. — URL: <https://doi.org/10.14778/2735496.2735504>.
6. *Stanton I., Kliot G.* Streaming Graph Partitioning for Large Distributed Graphs. — 2012.
7. Fennel: Streaming Graph Partitioning for Massive Scale Graphs / C. Tsourakakis [и др.]. — 2014. — DOI: 10.1145/2556195.2556213.
8. Репозиторий ArangoDB. — 2025. — [Обращение: (21.09.2025)]. <https://github.com/arangodb/arangodb>.
9. *Andersen R., Peres Y.* Finding sparse cuts locally using evolving sets. — 2009. — DOI: 10.1145/1536414.1536449.